Frankfurt University of Applied Sciences

Faculty of Computer Science and Engineering

Implementation and Evaluation of an Enterprise Architect Chatbot

Using a RAG-Based Approach

Thesis to Obtain the Academic Degree

Master of Science (M.Sc.)

Submitted by

Hendrik Gruber

Matriculation Number: 1458240

Advisor	:	Prof. Dr. Jürgen Jung
Co-Advisor	:	Dr. Rainer Schlör

Hendrik Gruber: *Implementation and Evaluation of an Enterprise Architect Chatbot-Using a RAG-Based Approach,* April 16th, 2026

ACKNOWLEDGEMENTS

This Master's thesis would not have been possible without the guidance and support of several individuals who contributed to this important milestone in my life.

A debt of gratitude goes to my Mom and Dad for enabling me to pursue higher education. You opened countless doors for me—support I can only hope to repay by continuing to give my best in life. Danke euch!

To my professor Prof. Dr. Jürgen Jung: thank you for your patience with me during my academic pursuit and for continuously pushing me to achieve more than I could have expected from myself.

Last but not least, thank you to my co-advisor Dr. Rainer Schlör for taking the time to support me during my thesis and for putting so much trust in me and my work.

DECLARATION (ERKLÄRUNG)

I confirm that this examination is my own work and that I have clearly referenced all tools and sources used. I have not made use of the work of any other student(s). Any sections quoted or paraphrased from other works are clearly marked as such. This examination has not been used for any other examination procedure of the Frankfurt University of Applied Sciences or another university in its entirety or in substantial parts. I understand that a plagiarism check may be carried out.

I used AI-based writing tools and/or other AI applications to write my assignment. I used these tools for guidance only. I confirm that most of the assignment is in my own words. I have listed as tools all AI applications used, including their product name and source (URL), and provided a complete overview of all functions used for the purpose of writing this examination. I have marked all texts and sections which were generated by and copied from AI-based writing tools or other AI applications accordingly.

Hiermit versichere ich, dass ich die vorliegende Prüfungsleistung selbständig und ohne Benutzung anderer als der angegebenen Hilfsmittel und Quellen angefertigt habe. Inhalte aus Berichten oder Arbeiten anderer Studierender wurden nicht übernommen. Alle Stellen, die wörtlich oder sinngemäß aus anderen Werken übernommen wurden, sind kenntlich gemacht. Die Prüfungsleistung war weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens an dieser oder einer anderen Hochschule. Mir ist bewusst, dass eine Plagiatsprüfung durchgeführt werden kann.

Ich habe zur Erstellung der Prüfungsarbeit KI-gestützte Schreibwerkzeuge und/oder andere KI-Anwendungen verwendet. Bei deren Einsatz habe ich steuernd gearbeitet. Ich versichere, dass mein eigener gestalterischer Einfluss in der Arbeit überwiegt. Sämtliche von mir verwendeten KIAnwendungen habe ich als Hilfsmittel mit deren Produktnamen und Bezugsquelle (URL) angegeben und eine Übersicht der im Rahmen der Prüfungsarbeit genutzten Funktionen vollständig aufgeführt. Die von KI-gestützten Schreibwerkzeugen oder anderen KI-Anwendungen generierten Texte oder Textpassagen, die übernommen wurden, habe ich gekennzeichnet.

Frankfurt am Main, April 16th, 2026

Hendrik Gruber

ABSTRACT

This thesis addresses the challenge of efficiently managing architectural information in Enterprise Architecture Management (EAM), where heterogeneous documentation and complex relationships require significant manual effort to extract and maintain. It introduces a Retrieval-Augmented Generation (RAG)-based chatbot named "Masutā", designed to assist enterprise architects in extracting and understanding proprietary architectural data while maintaining transparency. Masutā connects to a knowledge graph containing both textbook-based EAM knowledge and user-uploaded enterprise architecture data. Natural language prompts are translated into database queries, and the retrieved information is exposed to the user along with the query used to augment the result. This increases the transparency of generated answers and reduces the black-box characteristics commonly associated with Large Language Model (LLM)-based systems.

Action Research is employed as the research methodology, enabling iterative development of the prototype in close collaboration between the researcher and two expert practitioners. Across four development cycles and a final qualitative review workshop, the system was evaluated regarding functionality, usability, answer quality, transparency, and alignment with practical EAM requirements.

The research demonstrates that transparent natural language interaction with a proprietary EAM knowledge graph is technically feasible. At the same time, the evaluation reveals limitations of the developed system regarding prompting robustness and operational maturity. This thesis contributes an implemented and empirically evaluated grey-box RAG architecture for EAM and highlights transparency, schema awareness, and practitioner involvement as decisive design principles when integrating LLMs with enterprise architectural data.

CONTENTS

I THESIS

1 INTRODUCTION 2

2 TERMINOLOGY AND TECHNOLOGY 5
- 2.1 Enterprise Architecture Terminology . . . 5
  - 2.1.1 Enterprise Architecture Management . . . 5
  - 2.1.2 Enterprise Architect Role . . . 5
  - 2.1.3 Architecture Diagrams . . . 6
- 2.2 Technology . . . 6
  - 2.2.1 Large Language Models (LLM) . . . 6
  - 2.2.2 Retrieval-Augmented Generation (RAG) . . . 7
  - 2.2.3 Graph Databases, Neo4j, Cypher, and Knowledge Graphs 7
  - 2.2.4 Model Context Protocol (MCP) . . . 8

3 CURRENT STATE OF THE ART 10
- 3.1 PRISMA Framework . . . 10
- 3.2 Related Work and Research Landscape . . . 12
- 3.3 Related Projects . . . 18

4 ACTION RESEARCH 20
- 4.1 Action Research Design . . . 20
- 4.2 Action Research Setup . . . 21

5 CYCLE 1 24
- 5.1 Diagnosis . . . 24
- 5.2 Action Planning . . . 25
- 5.3 Action Taken . . . 25
- 5.4 Evaluation . . . 27
- 5.5 Learning . . . 28

6 CYCLE 2 30
- 6.1 Diagnosis . . . 30
- 6.2 Action Planning . . . 30
- 6.3 Action Taken . . . 31
- 6.4 Evaluation . . . 32
- 6.5 Learning . . . 34

7 CYCLE 3 36
- 7.1 Diagnosis . . . 36
- 7.2 Action Planning . . . 37
- 7.3 Action Taken . . . 37
- 7.4 Evaluation . . . 38
- 7.5 Learning . . . 38

8 CYCLE 4 40
- 8.1 Diagnosis . . . 40
- 8.2 Action Planning . . . 40
- 8.3 Action Taken . . . 41

8.4 Evaluation . . . 42
8.5 Learning . . . 43
9 FINAL REVIEW 45
9.1 Case Context . . . 45
9.2 System Setup and Data . . . 46
9.3 Review Execution Phases . . . 46
9.3.1 Phase 1 . . . 46
9.3.2 Phase 2 . . . 47
9.3.3 Phase 3 . . . 47
9.3.4 Phase 4 . . . 48
9.4 Interaction Procedure . . . 49
9.5 Observed System Behavior . . . 50
10 IMPLEMENTATION 53
10.1 Finished Architecture and Prototype . . . 53
10.2 Component Details . . . 54
10.2.1 Frontend . . . 55
10.2.2 Backend . . . 55
10.2.3 OpenAI . . . 58
10.2.4 MCP Server . . . 58
10.2.5 Neo4j Databases . . . 58
10.2.6 Local Environment via Docker . . . 59
10.3 Filling the Databases . . . 60
10.3.1 Creating the Textbook and SpeedParcel Knowledge Graphs 60
10.3.2 Creating a Custom Knowledge Graph via the XML Parser 62
10.4 Data Flow . . . 62
11 RESULTS AND DISCUSSION 64
11.1 Knowledge Generation and Transparency Mechanisms . . . 64
11.2 Architectural Limitations . . . 66
11.3 Make or Buy - Comparing Masutā to Rovo . . . 67
11.4 Practical Applicability in EAM . . . 69
11.5 Learnings From the Implementation . . . 70
11.6 Overall Goal Fulfillment . . . 71
12 CONCLUSION AND FUTURE WORK 73
12.1 Future Work . . . 74

BIBLIOGRAPHY 76

II APPENDIX
A INSTRUCTION MANUAL FOR MASUTĀ 81
A.1 Downloading Masutā . . . 81
A.2 First Time Setup . . . 81
A.2.1 Setting Environment Variables . . . 81
A.2.2 Preparing the Databases . . . 82
A.2.3 Setting Up the Databases . . . 82
A.2.4 Hard-Resetting Everything . . . 83
A.3 Running the Application . . . 83

A.4 Feature Overview 83
A.4.1 Input Field and Chat History 83
A.4.2 Toggling the Database 84
A.4.3 Resetting the Playground Database 84
A.4.4 Importing Custom Data 84
A.4.5 Inspect Database 84
A.4.6 Chat Context and Building on Previous Answers . . . 84
A.4.7 Exported Chat Protocol 85
A.5 Exporting XML Files From ArchiMate 85
A.6 Using a Different LLM 85
A.7 Example Questions 86
B MASUTĀ CODE SNIPPETS 87
B.1 Dynamic Schema Extraction and Injection for Query Generation 87
B.2 Schema-Aware Text-to-Cypher Translation via Prompt Engineering 87
B.3 Transformation of XML into a Neo4j Knowledge Graph 89
B.4 LLM-Based Generation of Contextualized Natural Language Responses 90
C MASUTĀ EXAMPLE QUESTIONS AND ANSWERS 92
C.1 What does HIS do? 92
C.2 What is an Orphan? 93
C.3 Which applications supports the most capabilities? 93

LIST OF FIGURES

Figure 3.1 A flowchart of the applied PRISMA framework depicting how many sources were initially found with each search method and how many were left for the final analysis of the current state of the art. The diagram aligns with the version described by V. Mishra and M. P. Mishra (2023) [22]. 11

Figure 6.1 Venn diagram of the three identified question categories (C1 = Category 1, C2 = Category 2, C3 = Category 3), which the prototype must be able to answer. C1 represents explicit textbook information, C2 represents implicit information of data, and C3 represents data combining the two, requiring the understanding of hidden rationales. 34

Figure 10.1 An architecture diagram showing each component and how they interact with one another. The Docker icon (blue whale) also indicates if the component is containerized. 54

Figure 10.2 A screenshot of a subsection of the textbook's knowledge graph. It shows how different node types (e.g., documents and sections) are connected within their own sub-graphs. On the right-hand side is the metadata for a selected node. Notice the "text" label within the metadata where it is clear to see the actual contents of the textbook that derived this node. 61

Figure 10.3 A UML sequence diagram showing the main flow of data when a user prompts the system. 63

LIST OF TABLES

Table 3.1 Concept matrix of related work in the research landscape. 13
Table 3.2 Overview of related projects and their positioning relative to the thesis at hand. 19
Table 4.1 An overview of Action Research Cycles condensed into three parts per cycle as well as a summary of the final review workshop. 23
Table 9.1 Summary of the four phases of the final review workshop. 49

ACRONYMS

AI	Artificial Intelligence
API	Application Programming Interface
APOC	Awesome Procedures on Cypher
EAM	Enterprise Architecture Management
HTTP	Hypertext Transfer Protocol
IT	Information Technology
LLM	Large Language Model
MCP	Model Context Protocol
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
STDIO	Standard Input / Output
UI	User Interface
UML	Unified Modeling Language
XML	Extensible Markup Language

USAGE OF GENERATIVE AI

OpenAI's GPT-5 and GPT-5.2 (OpenAI, 2025) [28] were used to aid in the creation of this thesis as described below.

- In the literature review to help expand search terms and improve search quality by considering additional / similar keywords leading to better search results in Google Scholar and other search engines, as well as by leveraging the "Deep Research" functionality, which conducts a multi-step research on the internet.
- To summarize meeting notes, ideas, and protocols while organizing and writing about the Action Research.
- To assist in the general writing process. Factual information was not generated with the assistance of AI. Rather, AI was used to proofread, give feedback and ideas, and support in polishing the paper's academic writing style.

Part I

THESIS

1 INTRODUCTION

In the field of Enterprise Architecture Management (EAM), artifacts containing architectural information can require large manual effort to understand, extract information from, and update. This is due to the various data sources within an enterprise's architecture and the complexity of the connections between data. Information is scattered across heterogeneous diagrams and documentation tools. This requires the enterprise architect to manually sift through these sources to retrieve the information that they need. Once retrieved, they must attempt to understand the current state of the architecture and how different parts relate to one another. Finally, once the data is extracted and understood, it must be updated to achieve a specific goal. Extracting, understanding, and updating the architecture can be a daunting task, especially for novice enterprise architects, as it requires a detailed understanding of the architecture. This increased effort introduces the risk of inconsistencies, artifacts that are hard to maintain and scale, and hinders data-based decision making. [17]

Through recent advances in artificial intelligence (AI) and Large Language Models (LLMs), it has become possible to process natural language inputs and leverage this to solve tasks without major programming efforts. LLMs can interpret natural-language user inputs to generate structured responses. This is achieved by training the LLM on large amounts of data. [13, 35] In combination with retrieval mechanisms, such as Retrieval-Augmented Generation (RAG), it is possible to supplement the responses of an LLM with the information of a specific knowledge base. This augmentation enables the system to answer domain-specific questions by retrieving relevant information and using it as context for answer generation. [44] Such systems are often implemented as chat-based interfaces, allowing users to interact with the stored knowledge through natural conversation. This opens the possibility of supporting enterprise architects in navigating and understanding complex architectural landscapes, reducing manual effort, and increasing the speed at which data can be understood.

Academic research has extensively explored integrating LLMs with structured knowledge sources, such as knowledge graphs, to reduce hallucinations and improve answer quality compared to answers generated solely from the model's pretrained data. Furthermore, research highlights the importance of grounding LLM outputs in domain-specific data when applying them to critical fields. [19, 34] However, there is limited work that demonstrates how such approaches can be applied specifically within the domain of Enterprise Architecture Management using proprietary architectural data and evaluated in a real-world setting. In particular, existing research rarely addresses how transparency can be maintained when an LLM dynamically

interacts with enterprise architecture data. Also, there is little documented evidence of how such a system can be validated together with domain experts.

Commercial solutions also exist. However, these solutions generally come with the trade-off of being operated by third parties, leaving control up to these providers. Because the details of such a system are rarely open source, the system becomes a black-box, disabling an enterprise architect from understanding how the tool reasoned, which data was retrieved, and how the final answer was generated. This lack of transparency can be particularly problematic when dealing with proprietary and business-critical architectural information. Without transparency, the line between retrieved facts, inferred relationships, and hallucinated content by the LLM becomes increasingly difficult to distinguish.

What is currently missing is a transparent, RAG-based chatbot, backed by a proprietary EAM knowledge graph, that explicitly exposes its retrieval and query mechanisms and is developed and evaluated in close collaboration with EAM practitioners.

Thus, the goal of this thesis is to develop and evaluate a prototypical RAG-based chatbot that supports junior-level enterprise architects in extracting and understanding architectural data better, allowing them to update the architecture without the otherwise required detailed understanding of the landscape. The chatbot is connected to a knowledge graph that has textbook information on EAM stored within, enabling the LLM to act as an EAM expert. The knowledge graph also contains the proprietary enterprise architecture data. A user can interact with the chatbot and ask questions about the stored data. A key feature of the chatbot is the aspect of transparency, which is ensured by allowing the user to examine the section of the knowledge graph used by the LLM to augment its answer.

Action Research was applied as the methodological approach for this thesis, as this enables an explorative development approach while maintaining academic integrity. This allowed the researcher and two expert practitioners to iteratively develop the prototype and guide the development in the right direction by assessing the system's functionality, usability, answer quality, and alignment with practical EAM requirements in each cycle. After the last iteration, a final review was conducted to evaluate the system's state and whether or not the goal of the research was achieved with the prototype.

This thesis contributes to the research domain in various ways. On a technical level, it demonstrates how a RAG-based chatbot can be implemented using a proprietary EAM knowledge graph and how LLM-generated queries can be used to extract relevant information from it. Furthermore, it illustrates how retrieval and query mechanisms can be exposed in order to increase transparency and reduce the black-box characteristics commonly associated with LLM-based systems. From an EAM perspective, the research highlights the importance of transparency and practitioner involvement when applying an LLM to an enterprise's architectural data to maintain integrity within the

architecture. The research also shows how such a system can be iteratively developed and evaluated in collaboration with domain experts.

This thesis is structured as follows. Chapter 2 defines the key concepts used throughout the research. Chapter 3 details the research methodology and provides an overview of current research and implementations within the domain. Chapter 4 describes what the Action Research method is, while the subsequent chapters 5, 6, 7, and 8 describe the details of each iteration of the development. Chapter 9 explains how the final review was conducted and what was observed. Building on the results of the development cycles and final review, chapter 10 explains the architecture and technical details of the final prototype. Chapter 11 then discusses the results of this thesis and what the results imply. Finally, chapter 12 presents the conclusion and possibilities for future work.

This thesis assumes the reader understands general terms in the realm of application development and EAM. An intermediate understanding is sufficient, as all key concepts are explained in detail in the next chapter.

2 TERMINOLOGY AND TECHNOLOGY

This chapter goes in detail on the terminology and technology that will be relevant for the reader to have a foundational understanding of the rest of this thesis. Later chapters will build on these concepts and pieces of technology.

2.1 ENTERPRISE ARCHITECTURE TERMINOLOGY

The first section describes relevant EAM terminology. It gives a high-level overview of what EAM is, what the role of enterprise architects contains, and what typical architecture diagrams are.

2.1.1 *Enterprise Architecture Management*

Enterprise Architecture Management can be summarized as being the bridge between the business and IT departments of an enterprise. The goal is to implement information technology aligned with the company's business needs. This is in contrast to the IT department implementing information technology for the sake of implementing information technology, which people in IT are often fond of doing. [1] An unwanted situation would be when the IT department falls into a siloed way of thinking, becoming decoupled from the rest of the company. EAM helps to ensure that the implemented information technology achieves the right things, namely, supporting the business capabilities and processes. [17, pg.s 2-3]

Ensuring that the architecture aligns with business needs requires understanding the organization's business capabilities. Business capabilities are defined as what the organization does, in contrast to a business process, which describes how an organization does something. A business capability thus supports achieving the business strategy by enabling the organization in what it does. [17, pgs. 46-47]

Core concepts of EAM include maintaining application landscapes, ensuring business capabilities are being enabled through the enterprise's applications, and aligning IT systems with business objectives. The goal is to align IT and business by transforming the as-is architecture into an improved to-be architecture. [17, pgs. 14-25, 46-50]

2.1.2 *Enterprise Architect Role*

The role of an enterprise architect is thus complex, as they face the challenge of preparing, planning, deciding, and supporting the changes necessary to improve the application architecture to support the business. A core task of

the enterprise architect is creating the necessary artifacts by gathering the necessary information and presenting it in the relevant types of documentation. Applying changes to artifacts in a landscape involves making them public and accessible to the relevant stakeholders. The enterprise architect is also responsible for maintaining compliance regulations, as not everyone in an enterprise should be able to view this information. Because architectural landscapes are ever-changing, the enterprise architect is also faced with the task of maintaining the existing artifacts. This also includes establishing processes and mechanisms to stay on top of the changing requirements of the organization and its stakeholders. Lastly, the enterprise architect must organize this process. They are responsible for selecting the right tooling to manage the architecture, organize an EAM team, and embed enterprise architecture in governance mechanisms. [17, pg. 152-154]

2.1.3 *Architecture Diagrams*

Enterprise architects deal with various artifacts, many of which are architecture diagrams. Common architecture diagrams and artifacts include business capability maps, application landscapes, business support matrices, and business object models, to name a few. Each of these diagrams serves its own purpose in documenting an existing as-is architecture or an improved to-be architecture. For example, a business capability map gives an overview of the capabilities that an enterprise has. This map breaks capabilities down from a level 1 capability, which is more high-level, down to more detailed capabilities of level n. Application landscapes provide an overview of which applications are used throughout the enterprise, often grouped into functional categories. [17, pg. 53, 76] Combining the two results in a business support matrix, where the cross-section between an application and capability indicates whether or not the application enables the capability. This type of matrix allows enterprise architects to maintain an overview of which capabilities are covered by which applications, identify redundancies, determine which applications are being overly relied on, and uncover capabilities not covered by any application. [33]

2.2 TECHNOLOGY

The following descriptions of technologies will help the reader to later understand the implementation details. They serve as a high-level, but sufficient, description of core pieces of technology.

2.2.1 *Large Language Models (LLM)*

An LLM is a type of AI that is trained on large text datasets to understand and generate human-like language. LLMs are capable of supporting with language-related tasks where text needs to be generated, translated, summarized, analyzed, or questions answered. [13]

A major challenge when dealing with LLMs is their tendency to hallucinate. Since they are trained on broad and general datasets, mainstream models often lack domain-specific knowledge, and their limited specialized reasoning further amplifies this issue. At the same time, LLMs generate responses based on probabilistic patterns learned from these large datasets, meaning they are optimized to produce human-like text rather than to ensure factual correctness. As a result, their outputs can appear plausible and well-structured, while still containing incorrect information or flawed reasoning. [14, 38]

2.2.2 *Retrieval-Augmented Generation (RAG)*

Retrieval-augmented generation aims to reduce hallucinations by retrieving relevant information from an external database, such as a knowledge graph, and incorporating that information into the response generation process. [19, 44]

Although LLMs contain basic knowledge within the realm of enterprise architecture management, it may not be sufficiently broad enough or in-depth to answer domain- or organization-specific questions. Furthermore, generally trained LLMs do not have access to proprietary information about an enterprise's architecture. As a result, prompting a detailed question related to enterprise's architecture or organization-specific data can lead to hallucinations. [19] This is a known challenge that can be overcome by implementing RAG, as this allows domain-specific information to be augmented into the LLM's generated answer.

RAG offers increased flexibility, as the underlying database can be updated with more information without requiring the language model to be retrained. [44]

2.2.3 *Graph Databases, Neo4j, Cypher, and Knowledge Graphs*

A graph database is a special type of database that represents its data via nodes and edges. Via this structure, it is able to represent entities and the relationships between them. [32, pg. 1] For example, a single business capability may be represented as one node, an application as another node, and a relationship between the two with the information of how the application supports this capability. This structure enables graph databases to build models that closely resemble the real world and map closely to the domain at hand. [32, pg. 6 and 38]

A key feature of graph databases is the possibility to traverse the graph. As opposed to relational databases, where an index lookup would be necessary, graph databases allow neighboring nodes to be discovered. This is an advantage when attempting to find information related to a looked-up entity. [37, pgs. 34-35]

A popular, open-source graph database is Neo4j [27], which can execute queries in milliseconds. [32] A main advantage of Neo4j is its built-in fea-

tures to visually interact with the graph via its Neo4j Desktop application or web-based tool. [37] This gives the user the option to display the nodes and edges within the graph in an interactive interface. Finally, there are a variety of Neo4j libraries that extend its features. One such library is Awesome Procedure on Cypher (APOC), which extends Neo4j to allow specific procedures and functions. Relevant features include retrieving schema information and converting data. [24, 26]

Within this prototype, Neo4j is implemented as a standalone server and is accessed by the prototype via Cypher queries. [32, pgs. 27 and 77] Cypher enables access to the knowledge graph through specific patterns that match the data. Data can be added, read, and deleted via Cypher. Queries may match data exactly or specify a matching pattern to find similar or related nodes. This is done by specifying the pattern for traversing the graph and letting the database decide how to perform the retrieval. [32, pg. 27][37, pg. 49]

A simple example of Cypher can be seen in listing 2.1. This Cypher retrieves all capabilities and the corresponding applications that support them. This exemplifies the structure of how two nodes (application and capability) are connected via a relationship (supports).

```
MATCH (a:Application)-[:SUPPORTS]->(c:Capability)
RETURN c.name AS capability, c.id AS capabilityId
ORDER BY capability
```

Listing 2.1: A simple Cypher example to retrieve the capabilities supported by applications.

A knowledge graph is thus a graph database composed of various relationships to represent a real-world structure or network, as found in enterprise architecture diagrams. [39] Building such a knowledge graph and inferring information from it is a central concept within this research.

2.2.4 *Model Context Protocol (MCP)*

Modern AI applications are typically designed to connect to additional external tools and services, such as a Neo4j database, to allow the AI to perform actions directly via the application's Application Programming Interface (API). However, this brings the challenge that developers of the proprietary application must manually define interfaces to connect their AI application to these external services. If the AI application is to be connected with n external services, then it would normally need n custom-built connections; one for each service. This comes with a lot of overhead, makes interoperability difficult, and can hinder the long-term maintainability of a system. [2, 15]

This is where the Model Context Protocol (MCP), developed by Anthropic and released in 2024, comes into play. It standardizes the interface between an AI application and its connected external services by defining a structured protocol for tool discovery, invocation, and result handling. This al-

lows a secure, two-way connection between the AI application and external tools. [2] With an MCP server sitting between the MCP client (the AI application) and the external service, the MCP client only needs to communicate with the standardized MCP interface. [15]

The MCP server can provide an agnostic connection to external services, such as an e-mail provider or calendar application, by requiring the tool's providers to explicitly implement their services as MCP-compatible tool interfaces. [15] Many pre-built MCP servers are available that enable standardized communication with specific external tools. For example, Neo4j offers an official, open-source MCP Server. As described later in section 10.2.4, this is the MCP server implemented for the prototype within this research. [25]

3 CURRENT STATE OF THE ART

In order to understand how this research fits into the current state of the art, it is beneficial to summarize and review current research and implementations. This helps to scientifically position the research at hand and justifies its implementation methods. This chapter describes how research on the current state of the art was conducted and summarizes the most relevant papers and projects identified.

3.1 PRISMA FRAMEWORK

The importance of documenting the research process to demonstrate the exhaustiveness of the literature review conducted has been emphasized in prior work. A robust literature review comprises querying databases for relevant literature using keywords as well as performing backward and forward searches based on these findings. Because of the plethora of literature on the subject matter, systematically including and excluding existing research must be made as transparent as possible. Describing which existing research was included and excluded is vital for the credibility of a literature analysis. [5, 40]

To support in this, the Preferred Reporting Items for Systematic Reviews and Meta-Analyses (PRISMA) framework was applied. The goal of PRISMA is to show why the review was done, how studies were identified and considered, and what was found. [31]

Literature was primarily identified using the Google Scholar search engine. The search process involved defining and iteratively refining relevant keywords clustered around the main themes of this thesis. Forward and backward citation tracking was also applied to the extracted sources. Search terms included EAM, LLMs, RAG, and Conversational Agents. In addition to manual search methods, a number of additional sources were identified through prompted searches using ChatGPT, which served as a supplementary discovery mechanism. [28] A flowchart of the applied PRISMA framework can be seen in figure 3.1.

Setting up criteria to filter out excessive literature is necessary to leave only the relevant pieces. Academic, peer-reviewed sources were the main filtering criteria. This was to ensure the academic integrity of the research process and results. The research's focus was then laid on the categories defined by the keyword search terms. The sources found were required to provide value to this thesis. Given the practical, implementation-oriented nature of this thesis, particular focus was placed on design- and system-oriented contributions rather than purely theoretical or analytical studies. A further crucial filtering criterion applied was the publishing date, which is especially relevant for

research on LLMs and conversational agents, as models such as ChatGPT only adopted widespread use in late 2022 [6], meaning research before then is less likely to be applicable to this thesis. Metadata of a published paper, such as the number of times it has been cited by further papers, may also be an indication of how well grounded the source is within its domain. For example, a paper cited by thousands of further papers is likely to be a key paper within its domain. A paper cited by little to no further papers is to be critically considered. Non-academic sources were kept to a minimum.

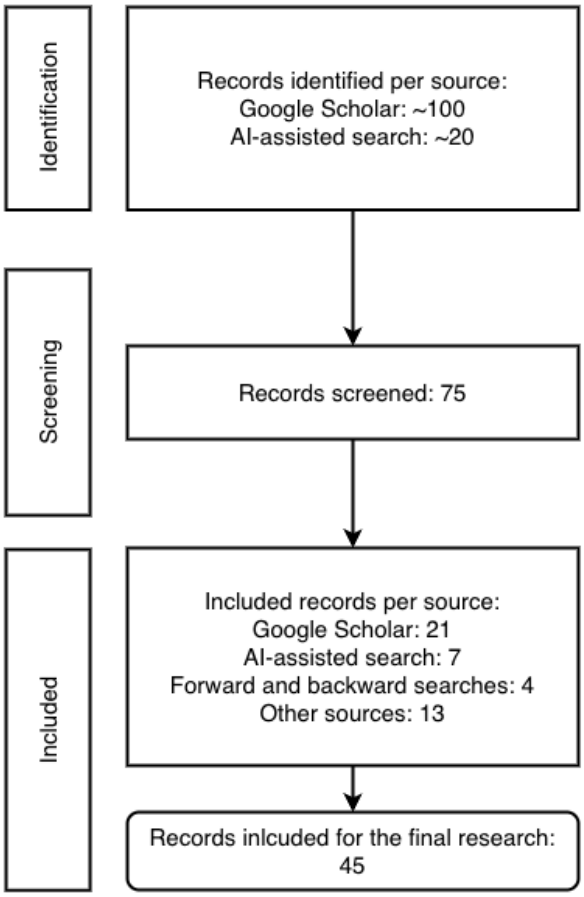

Figure 3.1: A flowchart of the applied PRISMA framework depicting how many sources were initially found with each search method and how many were left for the final analysis of the current state of the art. The diagram aligns with the version described by V. Mishra and M. P. Mishra (2023) [22].

After applying the above filters, the papers found were then initially skimmed to assess whether they might be relevant to this research. This involved reviewing abstracts and key contributions without yet analyzing the full content in detail. This allows papers that do not support the thesis's goal to be quickly filtered out. Afterward, the remaining papers were read more thoroughly by analyzing the applied methods, results, and contributions.

By applying the PRISMA framework as described above, the number of sources was reduced from an initial set of ~120 identified records to 75

sources after initial screening and finally to a subset of relevant academic sources forming the core literature base of this thesis.

In addition to the systematically identified literature, supplementary sources such as technical documentation and online resources were included during the development and writing process. These were not part of the PRISMA-based search but were included to support specific aspects of the developed prototype.

In total, 45 sources were used throughout this thesis. Of these, 21 were identified via Google Scholar, 7 via AI-assisted search, 4 via forward and backward citation tracking, and 13 from other sources such as technical documentation and online resources.

While the PRISMA process ensured a comprehensive identification of relevant literature, not all identified sources are equally suited for detailed analysis within this chapter. From the final set of 45 sources used throughout this thesis, only 7 of the most relevant papers and 3 related projects were selected for in-depth discussion within this chapter. These were chosen for their direct relevance to the research objective, particularly with regard to implemented systems and approaches that combine LLMs, RAGs, and structured knowledge representations. The remaining sources are used as supporting literature throughout the thesis, providing theoretical background and contextual grounding.

3.2 RELATED WORK AND RESEARCH LANDSCAPE

This section synthesizes and discusses the relevant literature and projects found through the previously described PRISMA-based research process. The selected works are semantically grouped and allow the reader to understand how the research at hand fits within the current research landscape. The purpose of this section is to identify what relevant research and approaches have been conducted to support the relevance and positioning of the developed RAG-based system of this thesis.

Table 3.1: Concept matrix of related work in the research landscape.

Source	EAM	LLM	RAG	KG	Query	Transp.	Key Contribution
Jung & Fraunholz (2021)	✓	x	x	x	x	~	Identifies key challenges in EAM, particularly lack of transparency in complex application landscapes.
Jung (2019)	✓	x	x	x	x	/	Highlights stakeholder requirements and recurring concerns in enterprise architecture practice.
Zhao et al. (2024)	x	✓	✓	x	x	✓	Introduces RAG framework and categorizes query complexity for LLM-based systems.
Wan et al. (2025)	x	✓	~	✓	✓	✓	Proposes schema-aware text-to-Cypher generation for domain-aligned querying.
Chen et al. (2025)	x	✓	~	✓	~	~	Introduces multi-hop graph retrieval for handling complex relational queries.
Laurenzi et al. (2024)	x	✓	x	✓	✓	~	Defines an LLM-assisted process for constructing and maintaining knowledge graphs.
Smajevic & Bork (2021)	x	x	x	✓	x	x	Presents automated transformation of conceptual models into knowledge graphs.

Legend: ✓= explicitly addressed, x = not addressed, ~ = partially addressed. EAM = Enterprise Architecture Management, LLM = Large Language Model, RAG = Retrieval-Augmented Generation, KG = Knowledge Graph, Transp. = Transparency

Jung and Fraunholz 2021 [17] lay foundational work for many concepts within the realm of EAM and identify recurring challenges associated with understanding, documenting, and managing enterprise architectures and application landscapes in practice.

One common challenge addressed throughout the textbook is that large, complex application landscapes are difficult to keep an overview of. In extreme cases, this can even lead to an enterprise lacking a complete overview of the applications in use, which severely limits transparency and hinders decision-making based on the data at hand. Another common hurdle arises from uncontrolled landscape growth, for example, through a merger or acquisition. In such scenarios, previously independent landscapes are fused into a single landscape, often leading to redundant systems, overlapping functionality, and unnecessary costs. Even in cases where the application landscape may not be overly complex, it may still suffer from poor documentation. This lack of reliable documentation hinders enterprise architects' ability to understand application purposes, dependencies, and the capabilities covered.

Jung (2019) [16] conducted eight interviews in which enterprise architects, consultants, and decision-makers in the German logistics industry identified concerns that enterprise architecture management is expected to address in practice. The results highlight that stakeholders address recurring topics, such as providing information about applications, optimizing application landscapes, and reducing the number of applications to save costs. These concerns provide a practical starting point for examining how enterprise architecture data can be structured, queried, and analyzed to support questions about existing landscapes.

The above expressed challenges can be summarized as a lack of transparency within the application landscape, and support the thought that these challenges can be addressed with the assistance of AI. Recurring tasks, such as retrieving simple information about an application landscape, can be challenging when done manually due to the complex, heterogeneous nature of large landscapes. Such hurdles may be especially difficult to overcome for junior enterprise architects, as they are not yet able to navigate the landscape. The challenges addressed by the previous two sources suggest they are predestined to be supported by AI-assisted tooling. This underlines the case for LLM-assisted EAM.

However, LLMs are notorious for hallucinating, meaning that incorrect information is spewed out while looking as though it could be factually correct. This is a major downside when interacting with an LLM, especially if expert knowledge in a specific domain is required.

Zhao et al. (2024) [45] investigate how to overcome this issue by supplementing an LLM with a RAG-based solution. Augmenting the context given to an LLM comes with the advantage that the data is up to date, aligned with domain expert knowledge, reduces hallucination, and the data supplied in the context explains the answers generated.

A key challenge the authors identified when developing a RAG-based LLM system is enabling the LLM to comprehend the varied forms of data required in the applied domain. In the case of the research at hand, this means that a central challenge when developing the RAG-based LLM system will be to structure and query the data so that the LLM has the right context to respond to the user with factual accuracy. It is also mentioned that making a system transparent helps increase trust and reliability in it.

To tackle this challenge, the authors identify four levels of complexity in information retrieval. The levels are as follows: level 1, for less complex queries; level 2, for moderately complex queries; level 3, for complex queries; and level 4, for complex information retrieval operations. [45]

Level 1 Explicit Facts: Require no reasoning and are facts found directly in the data.

Level 2 Implicit Facts: May require common reasoning or logical deductions based on the data.

Level 3 Interpretable Rationales: Require a grasp of the factual content as well as domain-specific knowledge, which is integral to the data's context.

Level 4 Hidden Rationales: Require that more information than that which is being asked is considered to generate the answer.

While queries at levels 1 and 2 are rather straightforward and only require the system to retrieve the relevant facts, queries at levels 3 and 4 require the system to learn and apply rationales that go beyond the data itself. The challenge of query tasks that require rationale can be overcome through in-context learning. The authors describe this as an LLM's ability to infer hidden rationales from relevant examples within the context of the prompt. [45]

These categories will be relevant again later in chapter 4 as these are also applicable within the domain of EAM. Each question posed to the system within this research can be assigned to one of these four categories.

However, categorizing questions does not yet address how information from a knowledge graph can be retrieved. This is especially challenging, as each question by a user contains a certain level of complexity that needs to be covered by the generated query. How can this challenge be overcome?

Wan et al. (2025) [38] investigate how LLMs can effectively be employed to generate Cypher queries and retrieve domain-specific information from a knowledge graph, in order to supplement LLM-generated answers by being more relevant to the domain at hand. The authors propose a text-to-Cypher approach that leverages a lightweight, semantic schema derived from the domain ontology. By injecting the relevant parts of this schema at query runtime, the LLM can be guided toward generating syntactically correct and semantically aligned Cypher queries without requiring model retraining and without overloading the LLM's context. The injected schema context enables the LLM to generate domain-aligned answers grounded in the structure of the knowledge graph.

This illustrates that a text-to-Cypher approach offers key advantages, such as aligning an LLM with domain-specific information and enabling domain-aligned reasoning without retraining the model. Additionally, this allows an LLM to remain up to date with a changing knowledge graph by dynamically incorporating the updated schema information at runtime. By exposing access to the knowledge graph through natural language inputs, the proposed approach lowers the barrier for non-technical users to interact with and extract information from it, allowing for the democratization of the information within it. The research by Wan et al. (2025) [38] further reinforces the suitability of a graph-based solution for the research at hand, as enterprise architecture involves highly interconnected data and requires reasoning across various entities and their relationships.

Combining the research by Zhao et al. (2024) [45] and Wan et al. (2025) [38], it becomes clear that a RAG-based LLM system should be able to assess the complexity of a user's request and generate queries accordingly. While

Zhao et al. (2024) [45] provide a high-level framework for understanding the complexity of the task given by the user, their work remains agnostic to concrete query-generation techniques required to interact with the knowledge graph. This gap is addressed by Wan et al. (2025) [38] through their text-to-Cypher approach.

Now, the question arises: how can a RAG-based LLM system be employed to retrieve the relevant information from the knowledge graph to answer the user's prompt with factual correctness?

Chen et al. (2025) [8] propose a graph-retrieval approach that is tightly integrated with an LLM by allowing it to iteratively retrieve and refine relevant subgraphs from a knowledge graph for downstream reasoning. This works by identifying seed entities within the graph based on the user's prompt. A graph retriever then iteratively expands from these seed entities along promising relations while pruning irrelevant nodes and edges. The extracted relevant subgraphs are then given to the LLM as context for reasoning. This multi-hop retrieval strategy allows the system to retrieve complex relational information necessary to answer more complex user queries.

In contrast to this multi-hop approach, simpler approaches rely on the LLM-as-retriever paradigm. In such approaches, the LLM performs a one-time extraction based on what the LLM considers relevant within the graph. Chen et al. (2025) [8] explicitly criticize this LLM-as-retriever strategy. They argue that this does not allow the subgraphs and their intricacies to be fully exploited. Moreover, it requires multiple LLM calls to explore different parts of the graph, leading to complex queries and scalability issues when applied to large knowledge graphs.

So far, it has only been discussed how to retrieve information from a knowledge graph. However, it has not yet been discussed how to persist information within it. Preparing and storing data in a graph representation poses its own challenges. The method for correctly breaking down data into nodes and edges depends on the source data. For example, converting a semi-structured textbook will require a different approach than converting an existing, structured application landscape. How can the challenge of creating a knowledge graph be overcome?

Laurenzi et al. (2024) [18] address this challenge by proposing a six-step process for creating a knowledge graph with the assistance of LLMs. The six-step cycle is as follows:

1. Understand what types of questions the knowledge graph will have to be able to answer, as defined in the previous research by Zhao et al. (2024) [45].

2. Construct the knowledge graph's schema.

3. Extract the knowledge from the data source and map it to the ontology of the knowledge graph.

4. Validate the mapping between the data source and the knowledge graph.

5. Add the extracted data to the knowledge graph.
6. Update and maintain the knowledge graph.

Their research takes an LLM-based approach to each of the six steps. For example, in step 1, an LLM was used in order to generate competency questions in natural language. In step 5, an LLM was used in order to generate queries based on natural language input. Within their research, ChatGPT achieved the best results. Following the example in step 1, the LLM generated well-tailored competency questions for the knowledge graph. For step 5, it was also able to generate queries, with only minor corrections necessary for successful knowledge extraction. However, the authors note that this step requires a certain awareness of the LLM and the underlying ontology schema, enabling the user to understand what information is being queried and how this may be improved. Query results can then be validated against the actual knowledge graph instance. This awareness is beneficial when working with LLMs and knowledge graphs.

The results of this study are beneficial for the research at hand, as it details how to set up a knowledge graph with the assistance of LLMs. This will be relevant again later. However, while the work by Laurenzi et al. (2024) [18] demonstrates how LLMs can assist in the creation of a knowledge graph, this process requires a human in the loop who is aware of the ontology. Moreover, this approach may be flexible, but it is not scalable. In many cases, when dealing with enterprise architectures, the data is already structured in architecture diagrams and can be exported from the respective modeling tool into structured formats such as XML. This raises the question of how an automated approach can be taken in cases where the artifacts exist and only require systematic parsing in order to be added to the knowledge graph, without the assistance of an LLM.

Smajevic and Bork (2021) [36] introduce a generic, end-to-end platform that enables automated transformation of conceptual models into knowledge graphs. The authors define a cloud-based platform that allows predefined models exported from Archi in XML to be uploaded. The uploaded model is then transformed and added to their Neo4j graph database. However, the authors do not provide detailed information on how this transformation is conducted. Relevant for the research at hand, the results show that an automated approach for transforming common export formats is a feasible solution for generating a knowledge graph using a preexisting conceptual model.

The above two studies by Laurenzi et al. (2024) [18] and Smajevic et al. (2021) [36] both exemplify two different approaches to creating knowledge graphs. While Laurenzi et al. (2024) propose an LLM-based, hands-on approach that offers flexibility in creating a knowledge graph from the ground up, Smajevic et al. (2021) offer a generic solution to transform an existing model into a knowledge graph in an automated fashion. Both approaches have advantages and disadvantages and will be relevant in chapter 11.

The next section steps away from academic papers and delves into similar projects and prototypes.

3.3 RELATED PROJECTS

Given that the research at hand concerns the development of a prototypical RAG-based LLM chatbot, it is worthwhile to briefly consider related projects. The goal of this section is not to provide a detailed description, but to position existing projects in relation to the prototype developed in this thesis. An overview of the related projects is shown in Table 3.2.

Edge et al. (2025) [12] introduce GraphRAG[1], a graph-based extension of RAG that addresses the limitation of traditional RAG systems in answering global, sensemaking questions over large document collections. The approach constructs a knowledge graph from the source data and generates hierarchical summaries that enable reasoning over the entire dataset rather than relying on a small set of retrieved documents. While this improves the comprehensiveness and diversity of answers, the method relies on precomputed summarization rather than precise, user-driven information extraction. As a result, it lacks direct query control and does not address domain-specific requirements such as enterprise architecture, limiting its applicability to the context of this thesis.

Dragon1[2] [11] is a SaaS platform for enterprise architecture management that focuses on modeling, analyzing, and communicating architectural artifacts through interactive visualizations. It emphasizes stakeholder alignment and transparency by providing a centralized repository and visual access to architectural data. However, it relies on manual modeling and exploration and does not provide AI-supported querying or automated access to architectural knowledge.

Opoku et al. (2025) [30] propose DO-RAG, a domain-specific RAG framework that combines knowledge graph generation and semantic retrieval to improve answer accuracy within specific domains. While the approach enhances domain alignment and reduces hallucinations through its four-step pipeline, it relies on a largely opaque retrieval process and does not provide transparent, schema-aware query generation. As a result, it does not fully address the need for controllable and inspectable information retrieval in enterprise architecture contexts.

All three approaches highlight different aspects relevant to this thesis. While GraphRAG demonstrates how knowledge graphs and LLMs can be combined to enable reasoning over large datasets, it relies on precomputed summarization and lacks precise, user-driven information extraction and domain-specific adaptation. In contrast, Dragon1 focuses on structuring and visualizing enterprise architecture data to improve transparency, but lacks intelligent retrieval and query mechanisms. DO-RAG, on the other hand, introduces domain-specific retrieval to improve answer quality but relies on an opaque retrieval process and does not provide transparent, schema-aware query generation. Similarly, recent advances in LLM-based text-to-Cypher

1 GraphRAG on GitHub: https://github.com/microsoft/graphrag
2 Dragon1: https://www.dragon1.com/

Table 3.2: Overview of related projects and their positioning relative to the thesis at hand.

Project	**Objective**	**Differentiation to this thesis**
GraphRAG	Global sensemaking over large text datasets using knowledge graphs and hierarchical summarization.	Focuses on precomputed summarization instead of precise, query-driven information extraction; lacks transparency and EAM-specific adaptation.
Dragon1	Visual enterprise architecture modeling and stakeholder communication platform.	Relies on manual modeling and exploration; no AI-driven querying or automated knowledge retrieval.
DO-RAG	Domain-specific knowledge graph generation and question answering using graph-based and semantic retrieval.	Lacks transparency in query generation; retrieval process remains largely opaque and not tailored to EAM-specific requirements.

approaches enable natural-language access to knowledge graphs but often fall short in terms of transparency in query generation.

This thesis builds upon these findings by proposing a schema-aware, transparent, and query-driven approach tailored to enterprise architecture data, combining the flexibility of LLMs with the required domain knowledge in EAM contexts. The next chapters examine the applied Action Research and the development of the prototype.

4 ACTION RESEARCH

The next six chapters describe how the chatbot “Masutā” (pronounced “mah-soo-taa” - a phonetic adaptation of the English term "master" into Japanese katakana) was developed. It provides further insight into the applied research method and the iterations gone through to create the prototype. This gives the reader the necessary understanding of how the research and implementation were conducted before discussing the implementation details afterward in chapter 10.

4.1 ACTION RESEARCH DESIGN

Action Research was applied in order to gain scientific value from the development of the prototype. The advantage of this research method is that it supports the development process for information systems. According to Baskerville (1999) [3], all types of Action Research share the following four characteristics: an orientation towards developing, a focus on a specific problem, an iterative process, and collaboration amongst participants. This is applicable to the work at hand because a prototype was developed to address a specific problem, in an exploratory manner, with support from industry experts.

Cyclical steps are central to the concept of Action Research, which contains an iterative process consisting of five steps within a single cycle. Other sources, such as Cornish et al. (2023) [10], propose variations with fewer (usually three) steps per cycle; however, these models also boil down to the same concepts. Across the literature, Action Research cycles follow the same structure: planning what to do in the new cycle, taking action, and evaluating the outcome of the completed cycle before moving on to the next one. [3, 10]

The paper at hand applied a cycle using the following five steps according to Baskerville (1999) [3]: diagnosing, action planning, action taking, evaluating, and specifying learning. The reason this research uses five cycles instead of three is to provide a more detailed description of the development process. This five-step cycle, including the preliminary and subsequent steps, built the basis of the conducted Action Research.

The following list describes the goal of each of the five steps [3]:

1. Diagnosing: Identifies the primary problems the organization needs to address.

2. Action planning: Researchers and practitioners collaborate to determine the next steps to address the identified pain points.

3. Action taking: The actual implementation of the planned actions.

4. Evaluating: The collaborative evaluation of the outcomes of the action-taking step. Evaluation includes determining whether the theoretical effects of the action were realized and whether this relieved the problems. The output of this step is generally the input for the next cycle.

5. Specifying learnings: The learnings of the cycle must be documented and applied to the next cycle.

Within this research, each cycle lasted between two and three weeks. The consulted sources do not mention how long a single cycle should last. However, a timeframe of two to three weeks per cycle was deemed reasonable for the development of Masutā because status updates were held at the end of each cycle, and with the given time for each cycle, there was enough progress to evaluate in these status update meetings. In total, four development cycles were conducted.

Action Research typically leaves the main theorizing up to the researcher. However, an extended form of Action Research, called Participatory Action Research, goes a step further by creating a more collaborative environment between the researcher and additional participants. Instead of leaving theorizing to the researcher, new information and ideas are generated together with the other participants, giving both parties an active role. This is beneficial because the other participants often have theoretical and practical knowledge of the subject matter being worked on. [3] From here on out, the term Participatory Action Research will be used interchangeably with Action Research.

The following subsections explain the design of the applied Action Research.

4.2 ACTION RESEARCH SETUP

This research was focused on the domain of EAM. In particular, this study primarily addressed application landscapes, business capability maps, business object models, and the relationships between these architectural artifacts. These artifacts are commonly used to document an enterprise's application landscapes and to align its IT with its strategic business objectives, as described in section 2.1.3.

Due to their structural complexity with heterogeneous data sources and multiple stakeholders involved, these artifacts are often large and difficult to interpret. Maintaining an overview can be especially challenging for junior-level enterprise architects. This challenge motivates the exploration of AI-supported solutions, which enable conversational interaction with the architecture rather than manual navigation of complex diagrams.

The research was conducted in close collaboration with the academic supervisor and the co-advisor. The academic supervisor gave academic guidance, supported the structuring of the research process to ensure academic relevance, and acted as an expert practitioner. The co-advisor acted as the domain expert and practitioner, contributing practical insights to ensure the

research has real-world relevance. The author of this thesis assumed the role of the researcher, implementing the Action Research cycles, validating findings, and planning subsequent steps in coordination with the other stakeholders.

At the outset of the research, the general problem was clear, but the potential solution was only vaguely defined. The co-advisor initiated the research with a vague vision of a centralized system containing all enterprise architecture information, which could be interacted with via natural language. The motivation for this was to reduce the effort required to interpret enterprise architecture artifacts and to offer alternative solutions to those available on the market.

However, in the early stages of the project, not only were the technical details of the potential solution unclear, but also the feasibility of such a system. Early on, it was mutually agreed upon that LLMs would play a key role in realizing this, even though the data structures, mechanisms, storage options, and interaction patterns were still open. Consequently, the initial problem statement was intentionally formulated at a high level to provide a suitable starting point for iterative exploration. Through iterative development cycles, this high-level problem statement, lacking a planned technical concept, was refined into a concrete problem definition, and a technical solution was developed.

Table 4.1 displays a condensed version of the applied Action Research cycles, including the final review. Each cycle is described in a condensed three-parts overview, giving a summary of what each cycle accomplished. The final review is also summarized at the bottom of the table.

Table 4.1: An overview of Action Research Cycles condensed into three parts per cycle as well as a summary of the final review workshop.

Cycle	Diagnosis	Action Taken	Learning / Outcome
Cycle 1	Technical feasibility.	Develop a proof of concept.	Built initial knowledge graph with textbook data. Hard-coded cyphers.
Cycle 2	Need to extend data and improve queries.	Integrate SpeedParcel dataset and improve querying.	Added SpeedParcel data. Replace hard-coded cyphers with MCP-based text-to-Cypher.
Cycle 3	Action Research identified as an appropriate methodology. Scalability and data format issues emerged.	Expand datasets via Archi models to prepare for XML imports	Recreated SpeedParcel models in Archi. Parsed XML.
Cycle 4	Over-reliance on LLMs for parsing and schema handling diagnosed as a critical limitation.	Replace LLM parsing with a generalized XML parser. Refactor schema handling. Prepare executable prototype.	Implemented APOC-based XML parsing and schema retrieval. Containerized local setup. UI enhancements added.
Final Review	A hands-on workshop with domain experts confirmed the feasibility and value of a transparent, RAG-based EAM assistant, while also revealing limitations related to context sensitivity, schema precision, and LLM-driven Cypher generation. The prototype was comparatively assessed against an industrial AI solution using realistic EAM questions, prompt variations, and data imports. The results highlight context dependency and over-reliance on LLM reasoning as key challenges. They also showcase the advantages of a custom RAG-based system, such as transparency, while also informing concrete lessons learned and directions for future research.		

It is important to note that during each cycle, continuous system tests were run by the researcher in order to ensure that the system's requirements were being met. For example, when adding new data to the knowledge graph, the system was prompted to test if it was able to retrieve this new data. In most cases, it was not able to do so right after adding new data. These tests led to the system's prompt needing to be continuously fine-tuned to handle the data in the knowledge graph. This was a routine step conducted regularly throughout development.

The development of the system was supported by generative AI tools such as ChatGPT. This was deemed appropriate as to accelerate progress in the prototype. [28]

With a clear understanding of what Action Research is and how it was set up for the research at hand, the next chapter will describe the first development cycle of Masutā.

5 CYCLE 1

With the beginning of the first Action Research cycle, development of the prototype commenced. The main goal of the first cycle was to produce a proof of concept for evaluation and to define the subsequent steps within the exploratory development approach. This chapter covers each of the five phases of the cycle.

5.1 DIAGNOSIS

The first cycle required several meetings and discussions with the academic supervisor and the domain expert in order to identify how an LLM-based system could support enterprise architects in their daily work. This initial vision lacked a concrete technical formulation, and the feasibility of natural language interaction with enterprise architecture data remained unclear. A vague end-goal was formulated without knowing how to achieve it or if it would be technically possible.

The co-advisor outlined typical enterprise architecture workflows and the tools used to document and interact with architectural artifacts in practice. An initial idea was proposed: an AI-based black-box system capable of ingesting architecture data and deriving its own internal representations. The idea of this approach would be to have an LLM-based system at hand which is able to read in architecture data, process it by building its own mappings and representations, and produce the resulting outputs autonomously. The achievability and academic applicability of such a black-box system were critically questioned, particularly because of the limited transparency of its inner mechanisms and the difficulty of testing them. Leaving 100% of the processing up to the LLM, without understanding how it performs this processing or how such a system would be engineered, was deemed an undesirable approach.

As an alternative, the academic supervisor proposed an explicit knowledge graph approach in which a knowledge graph is built and used to supplement LLM-generated answers via RAG. This differs from the black-box approach in that an explicit database is present, and the interaction between the LLM and the database is clearly defined.

The key distinction between the black-box and white-box approaches lies in their degree of transparency. While the black-box approach autonomously creates an internal representation of the information, the white-box approach requires the deliberate design and implementation of an explicit technical architecture. This transparency aligns well with the mentioned advantages of a RAG-based LLM system in section 3 by Zhao et al. (2024). [45]

These discussions were necessary in order to scope the solution space. Early visions of an end goal included a chatbot capable of supporting enterprise architects in exploring and improving application landscapes, for example, by identifying inconsistencies or incomplete architectural documentation. At this stage, the research methodology had not yet been explicitly defined as Action Research, and it was initially assumed that the resulting prototype would be evaluated through an expert interview.

With this broadly defined goal of a RAG-based implementation, the implementation phase of the first cycle began.

5.2 ACTION PLANNING

The goal of the first cycle was to create a proof of concept demonstrating the technical feasibility of a RAG-based LLM approach. This required setting up a system that supplements conversational interaction with access to a knowledge graph. In the first step, the knowledge graph was designed to consist of enterprise architecture knowledge grounded in textbook-based domain information. The reasons to use the textbook as the initial knowledge source were twofold. First, the textbook information was readily available, whereas an example enterprise architecture dataset was not yet accessible. Second, a key requirement of the system was that the chatbot should support decision-making within EAM. Consequently, the knowledge graph needed to contain domain knowledge about EAM and the responsibilities of an enterprise architect, both of which are readily available in the textbook.

Furthermore, a single-agent architecture was selected, as multi-agent architectures were considered unnecessarily complex for an initial proof of concept. Neo4j was chosen as the database system, and the textbook *Masterclass Enterprise Architecture Management* by Jung and Fraunholz (2021) [17], introduced in chapter 3, served as the primary knowledge source during this cycle. This approach corresponds to the research by Laurenzi et al. (2024) [18] described in section 3.2.

5.3 ACTION TAKEN

After setting up a test database within Neo4j, the knowledge graph was populated with data. The textbook consists of 213 pages of content in PDF format. This required the PDF content, consisting of chapters, sections, headers, paragraphs, figures, etc., to be decomposed into chunks that could be transformed into the corresponding knowledge graph structure of nodes and relationships. To support this task, the parsing service LlamaIndex with LlamaParse[1] was used during this research. LlamaParse can parse common document types, such as PDFs, and graphical representations of this data. On top of this, a GitHub repository by Joshua Yu[2] was used dur-

1 LlamaParse: https://www.llamaindex.ai/llamaparse
2 Joshua Yu's GitHub Repository: https://github.com/Joshua-Yu/graph-rag/tree/main/openai%2Bllamaparse

ing this research, which utilizes LlamaParse in conjunction with OpenAI's LLM to parse PDF documents, create embeddings, and save them to a Neo4j database. Minor code adjustments were required to align with the textbook's structure.

Once the knowledge graph was populated with textbook information, data validation was performed using Neo4j's interactive graph explorer. Samples of the knowledge graph were taken and compared with the textbook content. It was evaluated whether nodes contained information corresponding to textbook sections and whether nodes were connected to other nodes related to the same content.

With a viable knowledge graph setup, the surrounding system was developed to retrieve information from the database. A browser-based application was set up via React[3], which presented the user with an input field and the conversation history of the user's prompts and resulting responses of the LLM in a typical chat UI. The backend, written in Python, provided an endpoint for communication with the frontend and connected to the Neo4j database.

Information retrieval from the database was implemented using a fixed, parameterized Cypher query, into which the embedded representation of the user's query was injected as a parameter. This consisted of using a predefined embedding model to convert the user's input into an embedding, which is a numerical vector representation of the text. This embedding was subsequently used to retrieve semantically similar embeddings within the knowledge graph via a vector similarity search. The underlying idea is that if a user submits a prompt concerning business capabilities, then the embedding will be similar to other embeddings in the knowledge graph, which may contain relevant information about business capabilities. The top k embeddings were then returned, where different values for k were tested to strike a balance for returning just enough information from the knowledge graph to be able to answer the prompt without overloading the LLM with information. As a fallback mechanism, a simple keyword-based search of the user's prompt was implemented.

The raw information retrieved in this way from the knowledge graph is then fed into the LLM, allowing it to generate a natural language response, which is returned to the user. This allows the user to receive a natural-language response rather than raw information extracted from the database. This aligns with the prescribed goal of providing enterprise architects with a natural-language chatbot to assist their work.

The LLM used throughout this first research cycle was the Qwen2.5-7B-Instruct model[4]. The reason for choosing this model was that it allows a full local deployment without relying on an external model and it is free to use.

3 React: https://react.dev/
4 Qwen: https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

5.4 EVALUATION

The current state of the prototype contains the textbook information in a knowledge graph. The frontend and backend systems can take a user's input and retrieve relevant information based on the input's embedding. The LLM converts the retrieved information into natural language and presents it to the user.

The proof of concept at the end of the first cycle was demonstrated to both advisors and evaluated qualitatively through open discussion. Demonstration prompts showed that conceptual questions, e.g., on the definitions of EAM concepts, were answered satisfactorily by retrieving the correct sections of the textbook from within the knowledge graph. However, the satisfactory retrieval process was not applicable to all types of conceptual questions and left room for improvement in the consistency of information retrieval.

Both advisors assessed the approach as feasible, and the co-advisor confirmed that an explicit knowledge graph-based solution will be a viable direction for further development. The prototype successfully demonstrated technical feasibility in that EAM knowledge can be represented in a graphical structure, user prompts can be used for retrieval of information within the knowledge graph, and the retrieved information can be used as context in the LLM to generate domain-specific, natural language answers.

However, the current implementation has many pitfalls and leaves much room for improvement in later cycles. The first problem identified is the current querying method. While vector embedding is a viable method for retrieving information, it is less effective for explicitly structured knowledge in a knowledge graph. In the current data, everything is semantically structured and logically interconnected. In such cases, a deterministic Cypher-based retrieval approach may be more appropriate, as described by Wan et al. (2025) [38] in section 3.2.

Another challenge faced during the first cycle stemmed from the applied programming approach using generative AI. Although generative AI significantly accelerated the development of the proof of concept, it became evident that excessive reliance on it led to reduced code maintainability without continued AI assistance. This dependence quickly impeded further rapid development after the initial phase. This was an early warning sign that the further development of the system should not rely too heavily on generative AI.

The currently deployed Qwen model was also deemed too slow, with answers often taking minutes to process. This led to the idea of testing alternative options as the LLM of choice.

The last challenge arose from transforming the textbook information. At first glance, transforming a textbook into a knowledge graph appears to be a straightforward task. The inherent structure of a textbook (comprising chapters, subchapters, and paragraphs with logically separated ideas) suggests a clear mapping. A seemingly obvious approach would be to rep-

resent each chapter as a node, connect subchapters to their respective parent nodes, and attach individual ideas from paragraphs to the corresponding subchapter nodes. However, in practice, this transformation introduces several non-trivial challenges.

One such challenge lies in handling non-textual elements, such as tables and images, which do not translate directly into graph structures and require additional interpretation. A more fundamental issue arises from the size and sequential nature of the textbook. Since the entire content cannot be processed in a single step, it must be segmented into smaller chunks for analysis. This necessitates that the LLM processes each chunk independently and integrates the extracted information into the existing graph structure. In doing so, contextual continuity can be lost; for instance, a chunk may no longer retain a clear reference to its parent chapter or subchapter. As a result, ambiguities emerge regarding where newly extracted information should be attached within the graph.

An applied technique to mitigate this issue is to introduce a certain degree of overlap between consecutive chunks. This overlap helps preserve contextual continuity across chunk boundaries, enabling the LLM to better infer hierarchical relationships and maintain coherence when attaching information to the graph. However, this approach comes at the cost of introducing redundant information into the database, especially as the size of the overlap increases.

Not all of these challenges were fully resolved within the scope of this work. Consequently, the resulting knowledge graph does not perfectly capture every detail of the original textbook. This limitation is acknowledged and accepted as a trade-off of the chosen transformation approach.

The first cycle aligns closely with the six-step knowledge graph creation process proposed by Laurenzi et al. (2024). [18] Additionally, the initial scoping discussions among participants implicitly defined the different types of questions the system should be able to answer. The manual construction of the graph structure and data ingestion correspond to steps 2 through 5, including schema design via LlamaIndex, knowledge extraction from the textbook, validation via the Neo4j interface, and graph population.

5.5 LEARNING

Compared to the beginning of the cycle, where the feasibility of a centralized knowledge graph was uncertain, the first cycle demonstrated that an explicit white-box approach represents a viable path forward. The current version of the prototype offers a solid foundation on which subsequent iterations can further refine the technical direction toward the final system. At the same time, the cycle also revealed several core challenges that must be addressed throughout later development cycles. These challenges include transforming data into graph structures, extracting relevant knowledge from the graph based on a user's input, and generating responses by the LLM.

Furthermore, the first cycle highlighted the risk of relying excessively on an LLM for code generation, particularly when the core application logic is left up to the supporting language model. This dependency during development limits the maintainability of the code and contradicts the white-box philosophy, as this reduces the transparency at the code level.

Lastly, this cycle largely dealt with the challenge of breaking down the textbook into a knowledge graph. However, this challenge was unique to this cycle, as the textbook is now represented within the knowledge graph. Later changes to the knowledge graph will not require parsing textbooks or PDFs, but rather parsing architectural data. Consequently, the textbook parsing represented a one-time effort rather than a recurring task.

This first cycle served well as a starting point. The lack of clarity about which direction the project should take was resolved by the prototype. The first cycle has produced a prototype that can be built upon in the later cycles. The points for improvement identified in this cycle will serve as the input for the next cycles. It is therefore evident that further iterations are required to systematically address these challenges within an exploratory development process.

6 CYCLE 2

The second cycle builds upon the foundation laid during the first and marks the transition from a purely technical proof of concept toward a more structurally grounded and application-oriented prototype, thereby clarifying what the path toward a finished prototype might look like.

6.1 DIAGNOSIS

Following the initial feasibility assessment, the next identified challenge concerned extending the knowledge graph with additional information and positioning the prototype as a practically useful tool within the domain of enterprise architecture management. This required integrating the first elements of architectural data into the knowledge graph, as well as addressing the shortcomings of the existing querying method and LLM runtime performance.

One design consideration involved separating textbook-based domain knowledge from enterprise architecture data into two distinct databases. After discussion, a single integrated database was chosen to enable direct relationships between conceptual textbook knowledge and architectural artifacts. This decision reflects the assumption that conceptual and structural knowledge should not remain isolated, but instead mutually reinforce contextual reasoning within the system when answering a user's prompt.

The challenge imposed by the lack of real-world data to test the system was also identified. Potential test datasets were discussed to bridge the gap before incorporating proprietary enterprise data. To support development, it was agreed that the data from a completed university assignment from the Frankfurt University of Applied Sciences would be used, consisting of an application landscape, business capability map, business capability support matrix, business object model, and cross-application data-flow diagram. The dataset stems from a fictitious logistics company named *SpeedParcel*. Although fictitious, it provides a sufficiently complex architectural scenario to test structural integration and querying logic under realistic constraints. This SpeedParcel dataset will be used throughout the next cycles.

6.2 ACTION PLANNING

The objective of the second cycle was to extend the knowledge graph with additional data from the SpeedParcel dataset. This includes integrating SpeedParcel's business capability support matrix into the knowledge graph, in Excel format. To prepare this data for the knowledge graph, a parser will be written to transform the architectural diagrams into nodes and relationships.

A further objective is to improve the querying method. Alternative approaches were to be explored during this phase to evaluate whether the embedding-based retrieval method is appropriate or whether alternative retrieval strategies would prove more suitable. This exploration was guided by the shortcomings identified during Cycle 1, particularly regarding the graph querying method. In addition, alternative LLM models were to be implemented with respect to response quality, latency, and maintainability.

6.3 ACTION TAKEN

The transformation of SpeedParcel's capability support matrix was approached by developing a customized parser specifically written for the Excel file containing the matrix. The parser was developed by prompting an LLM with a description of the data structure and iteratively refining a Python script that reads the Excel file, decomposes it according to the predefined schema, and inserts the resulting nodes and relationships into Neo4j. This approach required iterative validation of the inserted graph structures to ensure consistency with the original architectural model. After several rounds of fine-tuning, the capability support matrix could be read into the knowledge graph.

To address the previously mentioned challenge of overreliance on an LLM when generating code, the researcher decided to rewrite the backend in Node.js[1], as the researcher is more versed in JavaScript backends than in Python. This rewrite did not change the application's core logic and still allowed typical client-server HTTP communication between the frontend and backend. However, this refactoring enabled a more maintainable codebase and improved transparency at the code level, thereby reducing the development's reliance on AI-assisted code generation. This decision also reflects a shift toward long-term architectural sustainability and scalability rather than short-term development speed.

During the backend refactoring, the idea for an MCP server emerged. Because Neo4j offers MCP support[2] via an APOC plugin, creating the MCP server and connecting it to the Node.js backend simply required executing predefined functions. This brought the advantage of a standardized interface between the application and the knowledge graph, abstracting direct database interaction.

However, the inappropriate querying method remained a challenge that needed to be overcome. An approach was tried out in which the user's input was used as context in a prompt towards an LLM, which is tasked with generating Cyphers to retrieve the information that the user is requesting. Rather than generating embeddings and performing similarity-based retrieval, the user's input was now used as contextual input for an LLM tasked with generating explicit Cypher queries. This text-to-Cypher approach requires that a system prompt be passed to the LLM as context,

1 Node.js: https://nodejs.org/en
2 Neo4j MCP: https://neo4j.com/docs/mcp/current/

supplementing the user's input with additional information and instructions about what the LLM is being tasked with. For example, the first version of the system prompt provides the LLM with information about the knowledge graph's structure, including the textbook's structure and the capability support matrix. It is also given basic instructions for the task of creating Cyphers based off of the user input for the specified knowledge graph structure. The LLM uses the system prompt alongside the user's input to generate Cypher queries tailored to the underlying graph schema. This approach shifts away from a similarity search toward graph traversal.

Implementing the MCP server along with the text-to-Cypher approach helps achieve a higher quality in the retrieved answers, as the user's prompt is now being transformed into custom Cyphers with the help of the system prompt. This allows the Cyphers being used to query the database instead of filling out the parameters of a hard-coded Cypher as before. The MCP server brings the advantage of executing these Cyphers along a standardized interface, rather than by connecting the backend directly to the database and executing the Cyphers directly on the database.

Lastly, the Qwen model was replaced with the latest GPT-5.2[3] model from OpenAI. This change required replacing all calls to the previous model with calls to OpenAI's API. The advantage of using such a mainstream model is increased speed and quality of generated answers, as well as improved development support due to more documentation and a higher expected lifetime of the model. However, this shift introduces per-request operational costs and reduces full local deployability. This trade-off reflects a prioritization of response quality and development reliability, as this research is concerned with developing a mature prototype rather than a potentially economically viable product.

6.4 EVALUATION

The more mature prototype was again evaluated qualitatively through a demonstration by the researcher and open discussions between all three stakeholders. Both advisors strongly approved of the direction taken and confirmed that the prototype was evolving toward a practically viable solution to achieve the end goal of enabling enterprise architects to interact with a proprietary knowledge graph via natural language. The prototype successfully demonstrated improvements in both conceptual and structural question answering. Conceptual questions within the EAM domain were answered with greater consistency, while structural questions targeting the SpeedParcel dataset were handled more reliably. The results with the new text-to-Cypher approach, along with the MCP server, significantly improved retrieval precision compared to the embedding-based method from the previous cycle. The new LLM being used also generates answers within seconds, rather than minutes, and does so with a higher-quality natural language output.

3 OpenAI GPT-5.2: https://developers.openai.com/api/docs/models/gpt-5.2

A prerequisite for the Cypher generation to work is prior knowledge of the knowledge graph schema. At this stage, the schema is hard-coded in the backend, which limits scalability, as it will require adapting the system context to accommodate later datasets added to the knowledge graph. This hard-coded mechanism is noted as a potential bottleneck as it introduces a dependency that may limit the knowledge graph's scalability when dealing with more heterogeneous datasets further down the line.

It was during this evaluation that the practitioners suggested that Action Research be applied as the research method for the explorative development process. This was deemed fitting because the approach so far has, by design, strongly resembled that of Action Research. Adapting to formal Action Research simply meant introducing academic conditions into the approach, such as documenting discussions with the goal of reproducing the efforts within this thesis. Committing to Action Research also meant that the focus could continue to be laid on developing the prototype, as opposed to spending time and effort laying out how the system may be tested via a different methodology. This allowed for a longer development phase within the thesis timeframe, enabling a more well-rounded prototype to be pursued as the finished product. Applying Action Research also implies that practitioners are not merely responsible for evaluating the current implementation but are active co-creators, generating ideas for further evolving the system.

During the discussions, it was regularly mentioned that enterprise architects work on architectural artifacts in the application Archi[4], which, as a tool, allows enterprise architects to graphically represent enterprise architectures. This means that, moving forward, the system must be able to import and interpret enterprise architecture diagrams exported from Archi.

It was also during these discussions that a deadline for the prototype development was agreed upon. Alongside the deadline, a final review was scheduled, during which all three stakeholders will test and evaluate the system together. This also means that the prototype must be prepared for such a final review. The use case of a final review requires that a local deployment be set up to allow the other practitioners to have the prototype running on their local computers. It was agreed to use an approach with Docker[5], as this allows each system component to be added to an encapsulated container, enabling an operating system agnostic execution method of the prototype.

Furthermore, the practitioners and researcher identified three key categories of user questions that will be asked of the system. The first category consists of conceptual questions related to enterprise architecture principles, concepts, and best practices found in textbooks. The second category contains descriptive questions targeting concrete architectural elements and relationships. The third and final category contains integrative questions combining conceptual knowledge with specific architectural contexts. The Venn diagram of this can be found in figure 6.1, and example questions of these

4 Archi: https://www.archimatetool.com/
5 Docker: https://www.docker.com/

categories can be found in the appendix's chapter A and C. This also overlaps with the categories identified in section 3.2 by Zhao et al. (2024) [45], where explicit facts map to the identified category 1, implicit facts map to category 2, and the interpretable rationales and hidden rationales map to category 3. This categorization provides a structured evaluation framework for the final review, as it allows the system's maturity to be assessed along progressively complex dimensions of reasoning.

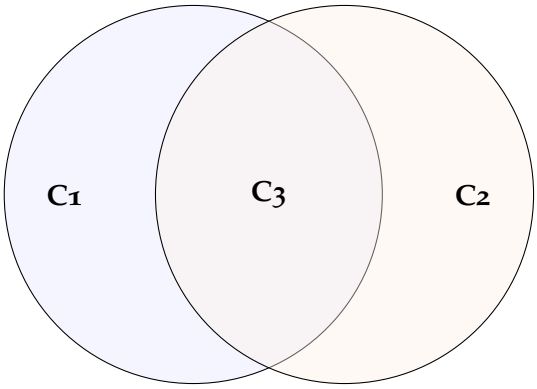

Figure 6.1: Venn diagram of the three identified question categories (C1 = Category 1, C2 = Category 2, C3 = Category 3), which the prototype must be able to answer. C1 represents explicit textbook information, C2 represents implicit information of data, and C3 represents data combining the two, requiring the understanding of hidden rationales.

The practitioners also expressed and envisioned practical use cases for the finished prototype, in which an EAM project plan is developed with support from the developed chatbot. For example, the chatbot may help identify which applications are redundant in an enterprise's architecture. Subsequent analysis of the business capabilities impacted by the removal or consolidation of these redundant applications can also be supported by the chatbot. This confirms that the prototype has progressed beyond experimental validation toward plausible practical applicability. Although the development is still exploratory, the system shows early signs of value creation within typical enterprise architect workflows.

6.5 LEARNING

As the prototype matured, clearer insights emerged from the second cycle.

Firstly, the previously implemented embedding-based retrieval method proved insufficient for dynamic interaction with the evolving graph structure. While similarity search may be suitable for unstructured textual knowledge, it became evident that structured enterprise architecture data benefits more from a traversal-based querying approach. The text-to-Cypher implementation demonstrated that generating explicit graph queries from the user's input enables the system to leverage the structural properties of the knowledge graph. This marks a conceptual shift toward a more controlled

graph traversal, reinforcing the importance of schema awareness when combining LLMs with structured data.

The switch to a mainstream LLM further highlighted the dependency on the underlying model. While the use of GPT-5.2 reduced full local deployability and introduced operational costs, it significantly improved response latency and output quality. This shift illustrates that developing a mature prototype may require prioritizing stability and performance over complete autonomy, especially when the objective is to evaluate conceptual viability rather than economic feasibility.

The introduction of the MCP server increased modularity and abstraction between system components, reducing coupling between backend logic and database interaction. This architectural decoupling represents an important step toward scalability. It became clear that as the knowledge graph grows and integrates additional datasets, clear interfaces between components are necessary to prevent the system from becoming rigid or overly dependent on hard-coded structures. The MCP server also allows further services than just Neo4j to be connected in a standardized way in future development.

Methodologically, the formal adoption of Action Research in this cycle clarified the collaborative nature of the development process. The practitioners did not merely evaluate the prototype but actively influenced its direction, for example, by introducing the requirement to support Archi exports and by defining structured categories of user questions. The categorization of conceptual, descriptive, and integrative questions also provided a clearer lens for assessing system maturity in subsequent cycles.

Finally, the broader discussions during this cycle revealed a gradual transformation in how the prototype is perceived. What began as an experimental knowledge-retrieval system is increasingly viewed as a potential decision-support tool for enterprise architecture projects. This shift in perception indicates that the system is moving beyond isolated technical experimentation toward integration within realistic architectural workflows. Although the development remains exploratory, the second cycle demonstrated that structured graph integration combined with controlled LLM orchestration provides a viable foundation for further refinement in the subsequent cycles.

7 CYCLE 3

Although the shortest, the third cycle marked a decisive phase in which the prototype began to take on its final structural form. In contrast to the previous cycles, which focused primarily on technical feasibility and retrieval quality, the third cycle shifted toward scalability, realistic data integration, and preparation for the final review. During this phase, the system moved further away from being an academic experimentation tool and closer to a deployable prototype intended to operate under practical constraints.

7.1 DIAGNOSIS

With a maturing prototype in place, the primary focus of this cycle was scalability and filling the knowledge graph with additional data. While the integration of the SpeedParcel capability support matrix in Cycle 2 had demonstrated that the approach was feasible, the system had not yet been tested against more complex architectural artifacts such as business object models and cross-application data-flow diagrams. Adding these further diagrams will allow testing the system across all three question categories, as identified and defined in the previous cycle.

During discussions with the practitioners, it became increasingly clear that enterprise architects predominantly work within modeling tools such as Archi, which allow them to export architectural artifacts in standardized XML formats. If the prototype is to be usable in realistic settings, it must be able to process such exports dynamically rather than relying on manually constructed datasets.

Another diagnosed challenge concerned the parser architecture itself. Up to this point, parsing had been implemented iteratively, per file. While this approach was sufficient for experimentation, it is not sustainable for a live demonstration or a final review scenario where unknown datasets may need to be imported without manual intervention. The risk of spending substantial time debugging or adapting parsers during practitioner meetings was identified as unacceptable for the envisioned final review.

In addition, a structural concern emerged regarding the size of real-world datasets. While the SpeedParcel dataset already contained several hundred nodes and relationships, real enterprise architecture models may contain thousands. This raised questions about parser robustness, querying performance, and overall system responsiveness under increased graph complexity.

7.2 ACTION PLANNING

The objective of the third cycle was therefore twofold.

First, additional datasets from SpeedParcel were to be recreated within Archi to simulate the realistic workflow of exporting enterprise architecture artifacts as XML files. As they stand, the SpeedParcel datasets are present in a graphical representation tool that is not meant for architecture diagrams and has no XML export option, requiring a recreation within Archi. In particular, the business object model (BOM) and the cross-application data-flow diagram were to be rebuilt within Archi and subsequently exported. This step ensured that the prototype would not merely process handcrafted data but rather data generated by a modeling tool that adheres to industry standards, as will be tested during the final review.

Second, the parsing logic needed to evolve beyond the current, file-specific, custom implementations. The goal was to move toward a more generalized XML parsing mechanism capable of interpreting structural patterns within Archi exports. This required analyzing the exported XML structure in detail and identifying stable mapping patterns between Archi elements, knowledge graph nodes, and relationships.

Parallel to these technical goals, the Action Research methodology was further formalized during this cycle. The previous development phases were retrospectively structured into explicit Action Research cycles, and documentation was aligned accordingly. This formalization reinforced the iterative logic of intervention, reflection, and adjustment.

7.3 ACTION TAKEN

Both the BOM and the cross-application data-flow diagram were recreated within Archi. These models were exported as XML files and analyzed in detail to understand their hierarchical structure and relationship encodings. Initial validation of the exported XML structure was conducted with the assistance of generative AI tools to assess whether the structural mapping into nodes and edges was conceptually feasible.

A further custom XML parser was then developed, again with AI assistance, but with a stronger emphasis on structural generalization rather than file-specific transformations. Instead of manually encoding transformations for each dataset, the parser was designed to interpret recurring XML structures and map them systematically to the graph schema.

During this process, several iterations of parsing and validation were conducted. The imported nodes and relationships were compared via samples in the Neo4j visualization and the original Archi diagrams to ensure semantic correctness.

In addition, the backend was optimized for context. The system prompt and schema descriptions were refined to better accommodate the newly imported datasets. This included adapting the contextual information passed

to the LLM during text-to-Cypher generation in order to reflect the extended graph structure.

All the while during testing, the generated Cyphers threw errors every so often. Usually, these errors were due to the LLM generating semantically incorrect Cyphers, which could not be executed. These error cases were used to fine-tune the system prompt by improving the description of the task assigned to the LLM to generate the Cyphers. With this additional context, the same errors did not occur twice.

7.4 EVALUATION

The progress of the third cycle was reviewed with both advisors through open discussions. The integration of Archi-based exports was assessed as a necessary and strategically important step toward realistic applicability. The overall development trajectory was again confirmed as appropriate, and it was agreed that the final cycle will focus on the necessary preparation for the final review.

However, the evaluation also revealed limitations. While XML parsing of the newly imported datasets was technically successful, the system struggled to consistently generate high-quality answers from the expanded graph structure. This was largely attributed to schema differences between datasets and insufficient contextual awareness within the text-to-Cypher generation process.

Furthermore, the increased graph complexity exposed potential performance constraints. Although no critical performance failures occurred, it became evident that the prototype had not yet been stress-tested under real enterprise-scale conditions. The gap between the SpeedParcel dataset and potential real-world datasets highlighted the need for additional robustness testing before the final review.

As this cycle was the shortest of the four, the goal of implementing a parsing structure that does not rely on a customized implementation per file was not met and will have to be completed as a priority in the last cycle.

Finally, it was agreed that the development phase would conclude after the fourth cycle with the structured final review. The requirement for the final review introduced a fixed deadline and further emphasized the need to achieve parser stability, system scalability, and a local deployment environment within the remaining timeframe. The fourth cycle would thus be the final development cycle within the realm of this research.

7.5 LEARNING

The third cycle revealed that the primary challenge was no longer feasibility, but scalability and robustness.

Firstly, the per-file LLM-assisted parsing approach reached its structural limits. While effective for experimentation, it proved unsuitable for handling diverse XML exports without manual intervention. This demonstrated that

LLM-assisted parsing is useful for rapid prototyping but insufficient as a long-term solution for standardized data ingestion. Moving forward, a more deterministic and schema-driven parsing architecture is required.

Secondly, the cycle highlighted the complexity introduced by heterogeneous schemas. This revealed that knowledge graph expansion must be accompanied by corresponding updates in contextual schema descriptions provided to the LLM. The scalability of text-to-Cypher generation is therefore tightly coupled with schema management.

Thirdly, discussions with practitioners reinforced that real enterprise architecture projects involve significantly larger datasets and more intricate interdependencies than those represented in SpeedParcel. This emphasized that performance and query optimization must become explicit design considerations rather than secondary concerns.

Finally, this cycle clarified the prototype's evolving identity. The system is no longer merely a knowledge retrieval tool, but is gradually approaching the characteristics of a decision-support system capable of assisting with architectural transformation scenarios. However, this transition requires stability, generalization, and scalability to ensure that the system can operate under realistic conditions without manual intervention.

In summary, the third cycle shifted the focus from improving answer quality to ensuring structural resilience. The insights gained during this phase directly inform the priorities of the fourth and final cycle.

8 CYCLE 4

The fourth and last cycle saw the prototype cleaned up and prepared for the final review. The focus shifted from adding new concepts to making the existing architecture robust and deployable. The final product is a shippable piece of technology.

8.1 DIAGNOSIS

The fourth and last cycle diagnoses that the system, in its current state, relied too heavily on LLMs for core processing tasks. Importing XML data into the knowledge graph is a critical point of the application, and relying on an LLM to handle this parsing was not considered a good choice. XML is a standardized format, and the export structure from Archi is sufficiently formalized to be handled deterministically. It was therefore agreed that a generalized parser should be used to parse incoming XML files into the knowledge graph, replacing the LLM at this critical point within the system's architecture.

A second critical issue concerned the querying method used to retrieve information from the knowledge graph. Although Cypher queries were generated by the LLM and tailored to the user's input, the approach struggled once the schema started changing due to additionally imported datasets. The main problem with this was the over-reliance on a hardcoded schema description within the system's context, which the LLM used as a reference for structuring the query. However, with an evolving heterogeneous knowledge graph, this approach became unreliable because the system implicitly guessed at the graph's structure rather than deriving it from the data.

Finally, the practitioners expressed a clear requirement for a deployment method that enables the prototype to run on any local machine. This was a prerequisite for the planned final review and required the system to become portable and the evaluation setup reproducible.

8.2 ACTION PLANNING

The goal of this cycle was to finalize the prototype and have it ready for the final review, where all three stakeholders would test and discuss the system together. Achieving this goal requires four concrete actions to be completed in the cycle. First, the XML data had to be parsed using a generalized, deterministic parser, replacing the LLM-based implementation. Second, the schema-handling strategy had to be refactored so that the system could dynamically understand the current knowledge graph structure at runtime, rather than relying on hardcoded schema descriptions. Third, a container-

ized local environment had to be set up to run the full architecture of the prototype on the practitioners' devices during the final review, with the exception of the LLM. On top of this, a controlled evaluation setup had to be prepared with various pre-filled knowledge graphs for the practitioners to experiment with.

8.3 ACTION TAKEN

The LLM-based data parser was replaced by an out-of-the-box solution from Awesome Procedures On Cypher (APOC). Instead of having an LLM infer nodes and relationships from XML, the system was refactored to load Archi-exported XML via deterministic APOC procedures. In this approach, the XML structure is parsed and transformed into Cypher-based graph insertions in a consistent and reproducible manner. This directly addressed the scaling issue identified in Cycle 3, as importing XML is a core requirement of the system and needs stable behavior across repeated runs and across different XML files.

The schema-handling strategy was updated away from the hardcoded variant. An intermediate implementation retrieved schema information via Neo4j's built-in call `db.schema.visualization()`[1] and injected the result into the system context before text-to-Cypher generation. While this improved results over a fixed-schema prompt, the output remained insufficient for robust query generation, particularly when multiple architecture diagrams with different structures were present in the knowledge graph.

This problem was also improved via APOC. Rather than using Neo4j's built-in schema call, the schema retrieval was changed to a predefined APOC function `apoc.meta.schema()`[2]. Compared to the previous schema call, the APOC call retrieves more detailed metadata about the schema, which is vital for the LLM's context. This ensures that the LLM understands how the knowledge graph is structured and how it may query it for the information requested by the user's prompt.

The local environment was prepared using Docker containers to support a reproducible execution setup on any machine. The containerized setup comes equipped with a frontend, backend, MCP Server, the necessary information to connect to an OpenAI model, as well as two local databases. The first database contains the SpeedParcel dataset, along with the textbook information, while the second database is a playground database pre-filled with textbook information only. This design created a controlled evaluation setup for two use cases: SpeedParcel provides a known structural reference dataset against which the prototype can be tested, while the playground environment enables importing additional XML files, thereby creating a proprietary knowledge graph.

1 Neo4j Schema Visualization: https://neo4j.com/docs/operations-manual/current/procedures/

2 APOC Meta Schema: https://neo4j.com/labs/apoc/4.1/overview/apoc.meta/apoc.meta.schema/

The user interface (UI) was also refined to support the final review workflow and to increase transparency. Improvements included the ability to upload XML files directly through the web application, toggling between the SpeedParcel and playground database, and adding a button to copy the generated Cypher query of each response, allowing the user to inspect the Cypher and raw data directly in Neo4j. Additional developer-facing convenience features were added, for example, quick access to connection information and opening the Neo4j console. This will reduce friction when opening and connecting to Neo4j during evaluation. A minimal form of conversation memory was also introduced by storing the previous user input and system answer, enabling follow-up questions to refer to the immediately preceding turn.

A practical challenge emerged once the system supported user uploads: resetting the database into the default state. A full restore from Neo4j backup files occurs at the container level and would require command-line operations by the user, which was not appropriate for the intended evaluation workflow. Several alternatives were explored, including dumping the textbook graph into Cypher scripts and replaying them into the database. However, this proved inefficient due to file size and runtime constraints. Therefore, the final solution implemented a reset mechanism that deletes all nodes and relationships that do not belong to the textbook, which is loaded by default, thus restoring the playground database while keeping the reference textbook intact.

Finally, portability was validated through a fresh installation outside the researcher's development environment. The prototype was installed on a separate machine, and the installation documentation was updated accordingly. This served as a practical test of deployment quality and reduced the risk of environment-specific errors on either practitioner's device during the final review.

8.4 EVALUATION

The refactored prototype was reviewed by both advisors through their own internal testing using the example data. The review focused on whether the system was executable, stable, and suitable for the final workflow. In particular, the practitioners assessed whether the prototype could import XML data, generate usable Cypher queries in the presence of a changing schema, and answer the three categories of questions defined earlier. The replacement of the LLM-based XML parser with a generalized APOC-based parser and the schema refactoring toward `apoc.meta.schema()` were assessed positively, as both changes increased reliability and reduced failures caused by schema changes. A major milestone of this cycle was that the prototype answered all three question categories consistently across repeated tests for the first time.

It is important to note that the custom parsing approaches developed in earlier cycles for transforming textbook content and the initial parts of the SpeedParcel dataset into a knowledge graph performed well during the

proof of concept phase. These implementations demonstrated that structured domain knowledge can be successfully extracted and modeled using tailored logic. However, they were tightly coupled to specific data formats and lacked the generalizability required for handling arbitrary user-provided input, which is a vital requirement for the final review. As the system's scope expanded to include dynamic XML uploads, this limitation became increasingly critical. The transition to a generalized XML parsing approach was therefore necessary to meet the system's requirements of robustness, reproducibility, and scalability. This design decision is also supported by existing literature, which emphasizes the importance of systematic transformations from conceptual models to knowledge graphs [18, 36].

The containerized deployment was also evaluated implicitly during this cycle's readiness check. The prototype stack was executed via Docker, the two-database setup was tested, and the user experience improvements were validated in the intended review flow by toggling the datasets, uploading XML, and copying Cypher into the Neo4j console. The database reset mechanism was considered adequate for producing repeatable evaluation conditions. Overall, the advisors confirmed that a "definition of done" had been reached for the prototype phase: the system was not meant to be endlessly expanded before evaluation, but instead to be stabilized and prepared for the final review. This goal was achieved at the end of the fourth cycle.

Beyond the technical assessment, the cycle review also produced forward-looking implications. The practitioners emphasized the value of local execution for compliance reasons when working with real-world data and discussed potential future deployment alternatives such as Podman[3] in work environments where Docker may be restricted.

8.5 LEARNING

Many learnings can be taken from this last cycle. The first is that replacing LLM-based parsing with a generalized XML parser improved correctness, stability, and reproducibility when generating a knowledge base and retrieving information from it. This matters specifically because importing data into the knowledge graph is a central requirement for a system intended to support enterprise architects in their daily work. If importing behaves non-deterministically, the resulting knowledge graph may cause problems downstream.

The second main learning of this cycle was that text-to-Cypher generation is only viable when the LLM has an accurate view of the current schema. In earlier cycles, Cypher generation appeared straightforward because the SpeedParcel schema was stable and explicitly known. Once user-imported data was added, the schema became heterogeneous and partially unknown at runtime. This made it clear that hard-coded schema prompts do not fulfill this requirement. The approach was therefore improved by retrieving the current schema directly from the database and injecting it into the system

3 Podman: https://podman.io/

context. Replacing Neo4j's built-in schema retrieval with the APOC equivalent improved the quality of the schema information and made Cypher generation more reliable across the playground datasets. A direct outcome of this refactoring was that the system could consistently answer all three question categories for the first time. This included conceptual, descriptive, and integrative questions of various kinds.

A remaining challenge that became very visible in this cycle is making Cypher generation independent of a specific data model. While schema retrieval reduces hardcoded contexts, it does not fully solve the problem. In order to navigate unfamiliar structures purely from the schema's meta information, the system prompt, and the user's prompt, the LLM is still tasked with in-context learning, as described by Zhao et al. (2024) [45] in section 3.2.

With the conclusion of this cycle, Masutā was born into existence as a finished prototype, ready to be tested during the final review. This is also where the stakeholders agreed that the definition of done had been met. Rather than endlessly expanding functionality, the work shifted toward freezing a stable, deployable state that can be evaluated fairly under reproducible conditions.

As described during the four development cycles, it becomes clear why Action Research was an invaluable methodology. From unclear beginnings containing only a vague vision of a final prototype, each development cycle contributed to the final architecture, allowing the end goal to become clearer and the development process more goal-oriented. Each phase helped examine what was technically possible and how the next iteration should be shaped based on practitioner feedback. This supported the exploratory nature of the project while still moving toward a defined artifact that is ready for the final review. In Cycle 4 specifically, this progress took the form of stabilizing the prototype around deterministic imports, schema-aware querying, and portable deployment. The next chapter examines how the final review was conducted and the lessons learned from it.

9 FINAL REVIEW

With all four development cycles completed, a hands-on workshop was conducted as the final review of Masutā. The purpose of this review was to test the prototype using real-world EAM questions to observe the system's behavior and identify its capabilities and limitations in a realistic setting. The review combined an exploratory testing approach with a summative evaluation and marked the formal end of the Action Research process within the scope of this thesis. This section focuses on describing the setup and procedure of the final review.

9.1 CASE CONTEXT

Masutā was put to the test by the two EAM domain experts who supported throughout the Action Research cycles. Their expertise ensured that the questions posed to the system reflected a real-world setting. The goal of the conducted review was to evaluate the final system end-to-end and compare its results with those of a similar, mainstream system named Rovo[1], which is integrated in Atlassian's knowledge management software Confluence. This setup allowed the domain experts to contrast the behavior and outputs of both systems under identical conditions. The conducted tests were done in a qualitative and explorative manner, meaning that the focus of the review was placed on the system's usefulness and perceived value from the experts' point of view.

This qualitative approach to reviewing the system was deemed appropriate, as the system was developed as a proof of concept to support enterprise architects in their daily work. A quantitative test scenario was not considered because the prototype was not developed with explicit quantitative requirements, for example, response times within x seconds. The practitioners interacted with both Masutā and Rovo in parallel while actively comparing results for similar prompts. The comparison was meant to reveal both the strengths and weaknesses of Masutā and to define what future development may add value to the system.

The final review was conducted as a four-hour online workshop between the researcher and both stakeholders, who acted as EAM domain experts, and contained a strict agenda. Parallel testing of two systems was conducted, accompanied by open, discussion-based feedback loops. Each participant interacted with the systems in an explorative manner.

1 Rovo: https://www.atlassian.com/software/rovo

9.2 SYSTEM SETUP AND DATA

The finished prototype system, as described later in chapter 10, was used throughout. Masutā was installed on both practitioners' computers. The system shipped with two databases. The first database included the SpeedParcel example dataset and textbook information. The second database served as an empty playground containing only the textbook knowledge. All components of Masutā, with the exception of the LLM, ran on the local environments of the practitioners' computers.

In order to simulate a real-world setting, a new set of data was introduced specifically for the workshop. The enterprise architecture for the examination office of Frankfurt University of Applied Science was used, which included several diagrams within Archi, ranging from high-level process landscapes to detailed views of examination preparation, thesis handling, certification assurance, and IT systems used by the examination office. Models were exported into XML format and read into Masutā's playground database as described in section 10.3.2 and A.4.4. The imported data from the examination office's architecture diagrams contained a total of 92 nodes and 152 edges. This dataset enabled evaluation in a realistic but controlled real-world scenario. During the system tests, the experts edited some of the faculty's architectural components with the generated answers.

Although Frankfurt University of Applied Science's architecture is described in German, prompts toward both systems contained a mix of both German and English. This reflects a real-world scenario in the German industry, where many companies employ both languages in daily work.

Parallel to the test cases applied to Masutā, Rovo was also being used with the same architecture data loaded into Confluence. Rovo can answer user prompts about the data via an internal, black-box knowledge system that it builds. This allowed the practitioners to test the same dataset with a second system. It included the same documentation and architecture data as found in Masutā's imported XML files.

9.3 REVIEW EXECUTION PHASES

The four-hour workshop was structured in four phases, each with a defined scope of experiments to be conducted on the systems. After a brief status overview in which each participant aligned on the current state of the prototype's development and the thesis, the first round of experiments began. A summary of the review can be seen in table 9.1, containing the goal and result of each phase as well as a summary of the closing phase.

9.3.1 *Phase 1*

The first phase served as a baseline check of whether both systems could handle simple, conceptual domain questions about the examination office and whether Masutā behaved as expected out of the box. The practitioners

asked short prompts such as "What does HIS do?" and "What is an orphan?" to test whether Masutā could retrieve relevant textbook context and ground its answer in the imported architecture model.

In this phase, it became apparent that wording and domain context strongly influenced answer quality. For example, when prompts explicitly framed the question in an EAM context (e.g., "In Enterprise Architecture Management, what is an orphan?"), Masutā tended to align more closely with the textbook definition, whereas shorter prompts increased the risk of a drifted definition.

The practitioners also observed that Masutā sometimes injected LLM-internal knowledge not present in the dataset, illustrated by a response that referenced a "QIS" system, even though it did not occur in the imported data.

In parallel, Rovo was tested with the same questions; the practitioners noted that Rovo also produced factually incorrect statements in some cases, which reinforced the need to treat both systems' answers as hypotheses that require validation.

9.3.2 *Phase 2*

The second phase shifted from conceptual questions toward direct interaction with the imported knowledge graph. The focus was whether Masutā could work with models that contained business functions with little to no accompanying documentation, and whether meaningfully structured outputs could still be derived. In practice, this phase was driven by prompts that required the system to summarize or reconstruct structures from the graph, such as generating a business support matrix and analyzing modeled elements around HIS, such as identifying that HIS-POS was explicitly present as a module. The practitioners also tried to operationalize outputs, for example, by requesting an Excel file export, which revealed a practical limitation in that the frontend did not support file download in the tested setup.

In addition, this phase exposed an important system boundary. When the practitioners asked the system to add descriptions to objects in the knowledge graph, then Masutā responded with read-only constraints, which produced Neo4j-related errors during the workshop. Overall, Phase 2 made clear that the system could generate structured, graph-derived summaries, but that interaction patterns must remain aligned with read-only behavior and output channels supported by Masutā at this stage of its development.

9.3.3 *Phase 3*

The third phase concentrated on quality issues in the imported model and on tasks that resemble real EAM grunt work, such as identifying missing descriptions, detecting missing objects, and validating completeness. The practitioners used prompts such as "Which business functions don't have a description?". This led to two observations. First, Masutā was able to list

business functions lacking descriptions and generate generic placeholder descriptions. Second, the practitioners judged these generated descriptions as too unspecific, which was then illustrated by the follow-up question "What is a TOR?", where the system correctly interpreted TOR as "Transcript of Records." This sequence highlighted that filling gaps via natural-language generation is possible, but that correctness and specificity must often be validated by experts.

This phase also included prompts to identify missing business objects in the context. In the observed output, Masutā defines missing as "existing in the overall model but not referenced by functions of that unit". Subsequently, the practitioner clarified that the intended meaning was "objects not yet listed in the specification."

Beyond the content-level findings, the practitioners reported that element typing mattered. For example, if the prompt did not provide ArchiMate terminology, such as "Business Function", the system often failed to retrieve relevant parts of the model and instead tended toward hallucinated or overly general answers. Guiding the LLM by prompting with the ArchiMate terminology improved the accuracy and relevance of retrieved information.

A further improvement point emerged around the generic import, as the practitioners suspected that too many nodes were imported under a generic label "ArchiElement" with the actual data type stored only in the metadata. This can reduce the LLM's ability to query the proprietary knowledge graph, thereby lowering the quality of the generated answers.

9.3.4 *Phase 4*

The closing phase consolidated findings and framed next steps without expanding the scope during the workshop itself. A central outcome was that the workshop format enabled a direct comparison between the transparent, inspectable system Masutā and the black-box system Rovo under identical data conditions. The practitioners explicitly emphasized transparency as a core differentiator: with Masutā, it was possible to inspect what the system executed in the background via the attached Cyphers with every response. Whereas Rovo's internal reasoning and retrieval remained opaque.

At the same time, Phase 4 captured the main limitations of Masutā discovered during the session. First, the system's sensitivity to prompt context and specialist terminology. Second, the risk of hallucinated knowledge that is not grounded in the imported dataset. Third, operational friction from read-only constraints when users implicitly request write actions. Lastly, instability symptoms were reflected in Neo4j error messages during some attempts. The practitioners also discussed concrete follow-ups for future development, such as exporting chat logs for documentation purposes and tagging the current GitHub state as a baseline version.

Finally, the workshop concluded with alignment on the broader narrative for the thesis: the system can incorporate expert knowledge via RAG, but it does not automatically infer the correct link between a user prompt and the

right slice of that knowledge without sufficient context; this was recorded as a key lesson learned and a driver for future architectural refinements, as described later in chapter 12

Table 9.1: Summary of the four phases of the final review workshop.

Phase	Goal	Result
Phase 1	Baseline check using conceptual EAM questions; parallel comparison with Rovo.	Answer quality depended on the prompt context. Explicit EAM framing improved alignment with textbook definitions. Isolated cases of hallucination through non-dataset knowledge were observed.
Phase 2	Test direct interaction with the imported XML model and the generation of structured outputs.	Analysis and summaries were possible. Export requests revealed frontend limitations. Prompts requiring write permissions to the database triggered read-only Neo4j errors.
Phase 3	Identify completeness and classification issues in the imported model.	Missing descriptions could be detected and auto-generated, but required expert validation. Interpretation of "missing" depended on phrasing, i.e., the LLM had a different definition of "missing" than the experts. Retrieval improved when ArchiMate terminology was used. The import structure of the generic ArchiMate-node is noted as a limitation.
Phase 4	The transparency of Masutā through inspectable Cypher was highlighted as a key advantage over Rovo, as well as the possibility to customize and extend Masutā with more features. Limitations documented include prompt sensitivity, hallucination risk, read-only constraints, and instability during response generation. Future enhancements and further features were discussed.	

9.4 INTERACTION PROCEDURE

This section describes the procedures used to interact with the systems, providing insight into the method applied. The prompts applied to both systems were structured to build on one another, covering all three categories of questions as identified in Cycle 2 in section 6.4. The prompts stretched from vague, short, and open structures to more contextually rich ones in order to test how the LLM deals with different context levels. Many prompts were refined iteratively during discussions. Both German and English prompts were applied to each system.

The interaction pipeline began with brainstorming questions to ask the systems, tailored for the current review phase. This required the experts to brainstorm questions to ask the system. These questions were then input into each system. The system then retrieved the information from their respective knowledge bases and presented the information to the experts. In the case of Masutā, the generated Cypher used to query the knowledge graph

was displayed alongside the response. In cases where the expert wished to understand the generated answer in more detail, they were able to copy the Cypher and execute it within their local Neo4j instance, returning the raw results used to answer the prompt. This allowed a deeper understanding of how the LLM came up with the answer. Rovo did not allow any further inspection into the generated answer.

Prompts were often entered into both systems in parallel, allowing real-time comparison of the outputs. After parallel prompting, the experts immediately began discussing the results. The researcher captured and documented observations throughout the session.

The answers generated were validated by the experts as soon as they were generated. This was done in one of two ways. First, by applying their own expertise, for example, when prompting the system for conceptual EAM knowledge, they compared it to their own understanding of EAM. Second, by manually comparing the generated results to the architecture of the examination office. Both validation methods were applied continuously throughout the experiments.

The following is an example list of prompts per category that were asked of the systems during the experiments:

1. Category 1: *What is an orphan?*, later refined to *In EAM, what is an orphan?*

2. Category 2: *Explain the business support matrix of the examination office.*

3. Category 3: *Which business functions don't have a description* with the follow-up prompt *Please create descriptions for all business functions.*

The prompts asked toward the systems differed largely from those asked during the development phase. While the researcher tested the system during implementation with typical questions, as in the examples above, the experts also regularly asked questions that went beyond the three categories. This included prompting the system to generate descriptions of architectures or to generate output files containing architectural information.

9.5 OBSERVED SYSTEM BEHAVIOR

This section describes the systems' behaviors and the responses they generated observed throughout the workshop. The focus lies on recurring patterns in outputs, system reactions, and interaction characteristics. A protocol of questions and answers between the experts and Masutā can be found in the appendix's chapter C.

Across multiple prompts, answer generation was strongly influenced by the level of contextual framing provided in the question. When domain-specific terminology, such as "Enterprise Architecture Management," or explicit ArchiMate element types, such as "Business Function," were included in the prompts, retrieval results more consistently referenced textbook definitions or correctly modeled elements. When such terminology was omitted,

responses were more likely to contain broader interpretations, generalized explanations, or even hallucinationed responses. For example, the conceptual question "What is an orphan?" generated different outputs depending on the phrasing. The shorter prompt led to an answer that explained that orphans are elements without relationships to other elements and referenced concrete orphans in the imported model. The more specific prompt "In Enterprise Architecture Management, what is an orphan?" led to the retrieval of a textbook-based definition of the term orphan, which defines an orphan as an application that does not support any business capabilities. This example can bee seen in the appendix's section C.3. While this difference in behavior is subtle, it illustrates that prompt framing directly affected which knowledge source was accessed. This can make a big difference when dealing with sensitive or proprietary data. Rovo was observed to be unable to generate factually correct answers and unable to derive a satisfactory answer to this question.

However, cases were observed where Masutā introduced information not present in the imported dataset. For example, when asked about the "HIS" system, the system referenced the system "QIS," although QIS did not appear in the underlying architecture model. This indicates that responses occasionally incorporated LLM-internal knowledge beyond the retrieved graph data.

When prompted to generate structured outputs, such as a business support matrix, Masutā required guidance in generating an appropriate output. After initial prompting, the system output an overview of the current business functions and processes. After the experts refined the prompt to generate a matrix, the system responded with a tabular representation in textual form, which highlighted unmapped functions. However, the output did not resemble a proper matrix of business functions and applications. Instead, the system output further analysis of the architecture, for example, that half of the applications did not have a business function assigned to them. After the expert further refined the prompts, the system was unable to fulfill the task properly and repeatedly went in circles, giving analytical responses to the data rather than generating a matrix. The expert requested that an Excel file be generated of the matrix. Although the system returned an attached file, the functionality of Masutā was not extended to support file handling. This means that the file may have been generated within the OpenAI servers, but was not able to be presented to the user within Masutā.

The expert then requested missing descriptions of the previously mentioned business functions. Masutā returned a list of business functions lacking descriptions, and recommended checking these for missing descriptions. After the expert requested that these descriptions be filled out and added to the architecture, Masutā responded with a Neo4j error that it is only able to execute read-only Cyphers. In contrast, Rovo did not generate any error messages in its outputs and produced answers without systemic errors in all test cases. The expert then requested the missing descriptions from Masutā via text, to which the system was able to output full-text descriptions.

Subsequent prompts aimed to improve the output format of the generated descriptions to facilitate easier copying and manual addition to the department's architecture in Archi.

A further observed limitation of Masutā regarded the Cypher generation process. It was observed that in some cases the generated Cypher did not solely retrieve existing graph elements but constructed a templated answer structure containing the requested keyword. For example, instead of retrieving the requested information from the knowledge graph, the generated Cypher contained hardcoded strings with an LLM-generated response directly in the Cypher. This meant that a Cypher was being executed within Neo4j without retrieving any information from the knowledge graph, giving the illusion that the necessary information was being retrieved. This manipulated response by the LLM could be detected through a manual analysis within Neo4j by copying and manually executing the Cypher returned along with the response.

The experts noted positively how Rovo generates its responses. Before outputting its results, Rovo often restated the question and its understanding of the information context before presenting an answer. This allowed the experts to verify whether the system understood the task correctly and to refine prompting when it misunderstood.

However, Rovo does not offer any further inspection of the generated results, such as how information was extracted from any sort of knowledge graph. It produces responses without exposing intermediate reasoning steps or retrieval mechanisms. This further illustrates the black-box mechanisms of Rovo. In some cases, Rovo also generated factually incorrect or misleading information, which was caught by the experts through manual analysis of the generated response.

The final review served as the concluding evaluation step of the applied Action Research methodology and formally ended the Action Research within this study. The final review also showed that the four development cycles were justified to deliver a working prototype, understand current limitations, and inform future improvements to the system. The next chapter will describe the technical details of the final prototype.

10 IMPLEMENTATION

The previous chapters answer the questions of how things were iteratively built, why they were built as they were, and how the final system was reviewed. This chapter contains all information about the finished Masutā prototype, including its architecture, individual components, features, data processing mechanisms, and a sequence diagram of how data flows through the application. Code snippets of critical parts of the final prototype can be seen in the appendix's chapter B.

10.1 FINISHED ARCHITECTURE AND PROTOTYPE

Figure 10.1 provides a holistic overview of the system architecture underlying the developed prototype. The user (1) starts the interaction by entering their prompt via a web-based React (2) frontend. The frontend serves as the primary interface for the user to interact with the application and its underlying data. Interaction is done through natural-language prompts. This makes up the presentation layer of the architecture.

Incoming requests are forwarded to the Node.js backend (3), which orchestrates the interaction between the frontend, the LLM (4), and the MCP Server (5). The LLM is used to turn the prompt into Cyphers and vice versa to turn the Cypher's result back into natural language. This is the only external service of the entire architecture and is thus not containerized. To enable standardized access to the Neo4j databases (6 and 7), the MCP server sits as a link between the application logic and the databases. These components constitute the architecture's orchestration layer.

The MCP server is connected to both Neo4j databases. The first is the playground database (6), which contains textbook data and the user's custom data they upload (8). The second database is the SpeedParcel database (7), which contains the textbook data and the SpeedParcel example data (9). Separating the two databases allows the user to toggle between the playground database instance, which contains their own data, and the SpeedParcel instance, which contains example data that is ready to be prompted. The databases make up the data access layer of the architecture.

Each of these application components is explained in more detail in section 10.2. With the exception of the OpenAI external service, each component has a Docker icon in the top-left corner. This indicates that the application component is part of the containerization. Each component runs in its own container.

Masutā in its final form is a RAG-based chatbot designed to support enterprise architects in querying and understanding complex architectural data through natural language interaction. At a high level, the system takes a

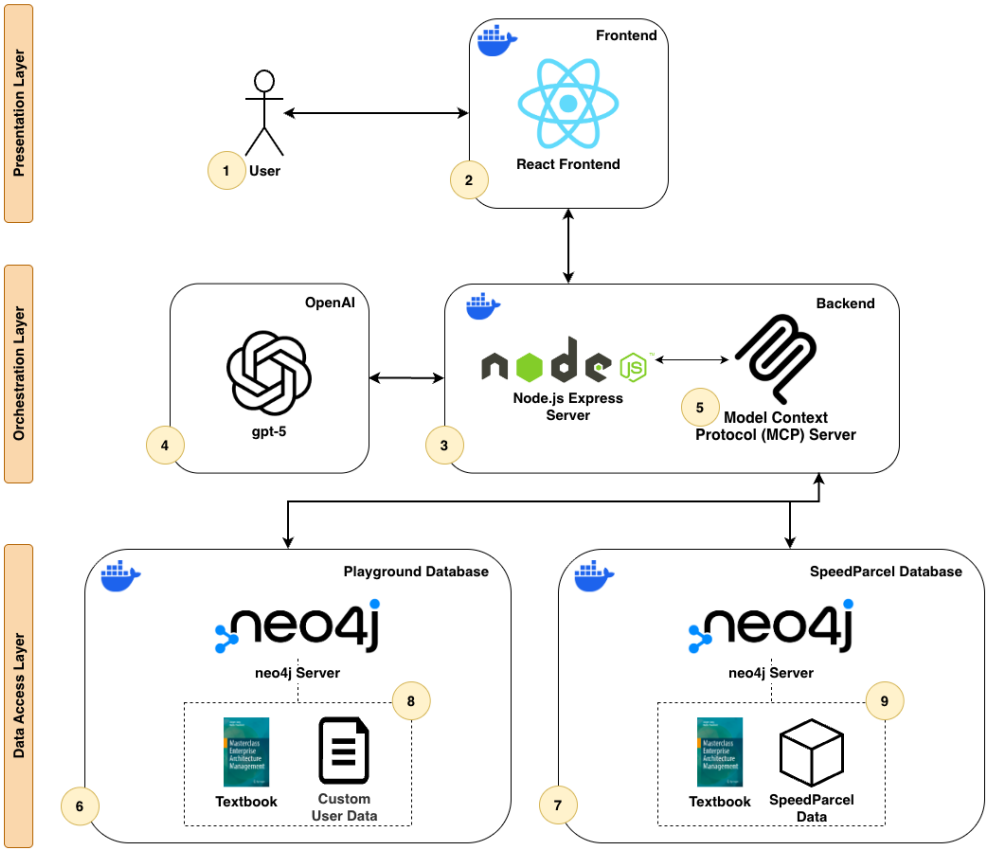

Figure 10.1: An architecture diagram showing each component and how they interact with one another. The Docker icon (blue whale) also indicates if the component is containerized.

user's prompt, translates it into a structured Cypher query using an LLM, retrieves the relevant information from one of two Neo4j-based knowledge graphs, and then transforms the result back into a concise, domain-specific natural language response. The prototype combines textbook knowledge with user-provided or example enterprise data, enabling both general EAM guidance and insights specific to an enterprise. A key characteristic of the system is its emphasis on transparency and verifiability, as the underlying queries and data sources can be inspected, allowing users to trace how each answer was generated.

10.2 COMPONENT DETAILS

This section describes each component in more detail, clarifying how and why each is implemented the way it is.

10.2.1 *Frontend*

The prototype's frontend is implemented as a single-page application using React and TypeScript. React was chosen due to its component-based architecture, which promotes modularity, reusability, and maintainability. TypeScript adds static typing to the JavaScript codebase, reducing runtime errors and improving development reliability in a growing application. [4] For the user interface, the Chakra UI component library is implemented. Chakra UI provides a set of accessible, pre-styled components that significantly reduce the need for custom Cascading Style Sheets (CSS) while ensuring visual consistency across the application. [7] This choice enabled rapid prototyping and allowed the development effort to focus on functionality rather than low-level UI styling concerns.

The frontend is containerized and exposed under `http://localhost:3000/`. It communicates with the Node.js backend through a set of RESTful HTTP endpoints. Each request includes the user's prompt and a session ID, generated when the page loads or when the user toggles the active database. The session identifier is used by the backend to maintain chat state and conversation history. By regenerating the session identifier on page reloads and database switches, the system ensures a clear separation of chat contexts. This prevents unintended carry-over of conversation history between different databases or interaction sessions, thereby preserving data consistency, analytical validity, and supports in maintaining better compliance.

Further details on the available frontend features and user example interactions are provided in appendix A.4.

10.2.2 *Backend*

The backend contains the entire application logic and is the most critical part of the prototype. Here, requests are received from the frontend, processed, then queried from the knowledge graph, and returned to the frontend. It also handles additional logic, such as parsing the user's uploaded XML files and inserting them into the knowledge graph, and supports features such as resetting the playground database. Many parts of the backend were developed with AI assistance. [28]

10.2.2.1 *Schema Information*

The ability of an LLM to perform a new task without explicitly retraining is called in-context learning. [41] This is done via a meta-prompt that is supplied to the LLM, where only a set of rules, inputs, or outputs are defined. The LLM then performs the necessary reasoning based on this context to complete the new task. This is injected into the LLM at runtime. [41] In this case, when the LLM is tasked with generating a Cypher for an unknown knowledge graph, the model uses the supplied context to infer the correct approach. In-context learning allows the LLM to learn how to generate the necessary Cyphers on the fly. [38, 41]

Before the LLM is able to generate any meaningful Cypher, it is necessary to retrieve the knowledge graph's schema. The schema is a machine-readable summary of the graph structure containing information such as the node types and their relationships to other nodes. This is done by calling `apoc.meta.schema()`. [26] Directly after getting the schema, the backend also queries the indexes present in the knowledge graph. This enables the database to look up nodes and relationships based on specific labels and properties. [23]

Code snippets for this can be seen in the appendix's section B.1. With this information, the LLM will later understand how the knowledge graph is structured and how Cyphers can be generated to query it.

10.2.2.2 *System Prompt*

The system prompt is one of the most crucial parts of the backend. This is where the LLM is given its information for in-context learning in order to understand how to transform the natural language inputs into a single Cypher and how to turn the raw Cypher response back into natural language for the response to the user. The implemented system prompts can be found in chapter B.2 and B.4 of the appendix.

The incoming user prompt requires a finely tuned, well-thought-out system prompt. The goal of the first system prompt is to provide the LLM with all the context that it needs in order to generate the specific Cypher required to fulfill the user's prompt. It contains four sections, categorized to help generate the Cypher. The first category in the system prompt is a set of rules on how it may generate the Cypher, e.g., to infer meaning from the user's prompt to identify which labels in the knowledge graph to query. The second category is generic schema information and the types of Cyphers it may use to fulfill different types of user requests. The third category in the system prompt is the task that the LLM is given, i.e., to generate exactly one Cypher, only Cypher, and no further text or explanations. The fourth and last category is a set of Cypher syntax rules. These were added iteratively during development as the generated Cyphers regularly had errors and required assistance to avoid these.

This system prompt, along with the schema information, the user's current prompt, including the potential previous prompt, and corresponding system answer, is then passed to OpenAI's GPT-5 LLM. The returned result is a single Cypher that can be directly used to query the knowledge graph in order to fulfill the user's prompt.

The second system prompt supports turning the Cypher's response back into natural language. It is simpler and contains only two blocks. The first block tells the LLM to act as a senior enterprise architect supporting a junior enterprise architect. The second block provides the LLM with guidance on how to structure the response, e.g., to keep the answer within 1-6 sentences, depending on the complexity of the question.

This second system prompt, along with the original user prompt and the Cypher's response, is again passed to OpenAI's GPT-5 LLM. The re-

turned result is then a natural-language response containing the answer to the user's prompt. This is the final result that is returned to the frontend.

Tuning these system prompts significantly alters the quality of the entire system's results. While the second system prompt is rather straightforward, the first system prompt for incoming user prompts is the result of fine-tuning during development in all four action research cycles, as described in section **??**. With less information in this system prompt, the generated Cypher's ability to retrieve the requested information worsens.

All of the above information is necessary for the LLM's ability to perform in-context learning specific to the knowledge graph's schema. Without this information, the LLM would not be able to generate Cyphers based on the user's prompt, nor would it be able to return an answer in an appropriate wording designed for an enterprise architect. Code snippets for the system prompt can be found in the appendix's section B.2

10.2.2.3 *Ephemeral Conversation Memory*

The prototype has an ephemeral conversation memory, meaning that once the session ends, the conversation's content is discarded. [43] This is where the session ID, which is generated and sent from the frontend, comes into play. The backend logic holds an array of past user prompts and system answers. These are tied to the session ID. An incoming request's session ID from the frontend is compared to the current session ID stored in the backend. If the session IDs match, then the previous user prompt and system answer are used as further context for the new user prompt. This enables the user to build upon their previous prompt or the system's previous answer, which may be necessary during complex, multi-step interactions. [43]

As the prototype stands, it saves only the most recent user prompt and the corresponding system response. However, this can be scaled to contain more of the previous prompts and responses. It was kept to a minimum for this prototype as this can bloat the prompt passed to the LLM, and quickly exceed OpenAI's character limit, increase the amount of time required for it to generate a response, and cause confusion and irrelevant outputs. [38]

10.2.2.4 *Resetting the Database*

The backend contains an endpoint that allows the user to reset the playground database via interaction in the frontend. This reset can only be applied to the playground database, as the SpeedParcel database is read-only and is not to be modified. The playground database is reset in such a way that only the user's uploaded data is removed and the textbook information is left remaining. This is done by deleting all nodes and relationships that are not explicitly part of the textbook data.

This covers all backend functionality. It is the central part of the prototype through which all data flows and all application logic is handled. The parsing and conversion of data is described in section 10.3.

10.2.3 *OpenAI*

The LLM used is made available by OpenAI and can be accessed via its API. [29] The LLM has two tasks within the prototype. The first being to transform the user's prompt into Cypher and secondly to turn the Cypher's result back into natural language.

The LLM is an external component and is not hosted, trained, or fine-tuned within the scope of this thesis. All interactions with the LLM were made via stateless API calls, where the user's prompt, predefined system prompt, schema, and conversation history were explicitly provided with each request. This allows the LLM to be decoupled from application logic and replaced by other models.

10.2.4 *MCP Server*

MCP is an open-source, standardized communication protocol that enables client applications to communicate with AI applications. The advantage of having an MCP server sitting between the backend and the Neo4j databases is the standardized interface it provides for communicating with external systems. [2, 15]

The MCP server employed in this prototype is the official Neo4j MCP server. It communicates with the Node server via MCP's STDIO (standard input/output) and includes standard functions for reading and writing data to Neo4j databases. [25] This allows the prototype's backend to query the databases without requiring proprietary code to do so.

10.2.5 *Neo4j Databases*

The need for two databases arose during development. Before real-world data was made available, the system required data during development and testing. Because the SpeedParcel data was readily available from a university course during a previous semester and already resembled a real-world enterprise architecture, it was used as a starting point for the prototype.

Both databases contain textbook information from the 2021 book "Masterclass Enterprise Architecture Management" by Jung and Bardo. [17] The knowledge graph, consisting of the textbook, contains 13.724 nodes and 13.525 edges. The advantage of having a knowledge graph of textbook information adjacent to enterprise data is that user prompts of category 3 can directly query both the textbook information and the specific enterprise data to supplement the generated response with information specific to the role of an enterprise architect.

The SpeedParcel database contains additional example data and serves as a starting point for users to explore the prototype and its capabilities. SpeedParcel's knowledge graph consists of 566 nodes and 788 edges. Within this knowledge graph are SpeedParcel's application landscape, business capability map, business capability support matrix, business object model, and

cross-application data-flow diagram. This data was read in via the method described in section 10.3.1 and 10.3.2. From the user's perspective, the SpeedParcel database is entirely read-only.

Out of the box, the playground database only contains the knowledge graph of the textbook information. It is up to the user to fill the database with their own enterprise data. This database can also be reset by the user from the UI, allowing them to start over.

Both databases can be interactively viewed either via the application "Neo4j Desktop" or via their web application. After connecting to the respective database, the knowledge graph can be queried via custom Cyphers and the results viewed in a visual and interactive way. Utilizing this tool along with the "copy Cypher" feature described in the appendix's section A.4.1 enables the user to inspect the raw information that the database returned before being transformed into a natural language response by the LLM. This allows the user to fact-check and retrace the answers generated.

10.2.6 *Local Environment via Docker*

During the later stages of development, the need for a fully local execution environment emerged to ensure that the prototype could be run on any machine, particularly in preparation for the final review. This requirement was fulfilled using Docker and Docker Compose. The central `docker-compose.yml` file in the project's root folder orchestrates the four containers required to run the prototype.

Both the frontend and backend components are built from individual Dockerfiles that define their respective runtime environments and startup commands. The backend defines a mounted volume, allowing the user's uploaded XML files to be persisted across container restarts.

The Neo4j databases were deployed using the official, predefined Neo4j image. The databases are initialized via two services defined in the Docker Compose, which must be executed ahead of starting the application for the first time. This initialization procedure is described in detail in the setup instruction in chapter A of the appendix.

As a result of containerization, the entire prototype can be started with a single `docker compose up` command. This containerization approach ensures a consistent environment across different host systems, thereby improving reproducibility and reducing environment-specific errors during evaluation. Furthermore, each component is encapsulated within a clearly defined architectural boundary, enabling independent scaling. For example, allocating more memory to a database can be adjusted independently without affecting other application components.

10.3 FILLING THE DATABASES

Filling the databases with knowledge graphs is a central part of this research. Establishing this method is the result of all four cycles of the applied Action Research. This section describes both approaches to filling each database.

The domain of Enterprise Architecture works well in combination with graph databases. Entities found in architecture diagrams can generally be stored as nodes, while the connections between entities establish the context of how two entities relate. With this structure, all of the architecture diagrams mentioned in chapter 2 can be depicted without losing information. [32, pg. 67]

10.3.1 *Creating the Textbook and SpeedParcel Knowledge Graphs*

At the beginning of the development phase, it was not clear what the end data would look like. One thing that was certain was that the textbook and SpeedParcel data would only need to be parsed once during development, allowing full data preparation before the user interacts with the application. This is in contrast to parsing the data while the user is interacting with the application. The difference means that the textbook and SpeedParcel data may be parsed with custom implementations, which are one-time endeavors, while the real-world data later has to be parsed with an ad-hoc, generic parser.

The custom parsers for the textbook and SpeedParcel datasets were implemented in Python using Jupyter Notebooks. The development of these was supported by the AI-based coding assistant ChatGPT from OpenAI. It was used as a support tool for tasks such as code drafting, refactoring suggestions, and troubleshooting, while the overall design, implementation decisions, and validation of the parsers remained the author's responsibility.

The first version of the parser was specifically designed to handle the textbook data in PDF format. This was a challenge that would only be faced once, as neither the SpeedParcel data nor the real-world data is found in PDF format. In contrast to XML, which inherently provides structure, PDF simply contains text and instructions for its representation. [20] Breaking this structure down into nodes and edges while attempting to retain the textbook's structure is thus a challenge.

To support parsing the PDF files, an open-source repository by Yu (2025), which combines OpenAI's API and LlamaParse, was forked and modified. [42] The changes made to the original codebase include specifying how to structure the textbook to keep the inherent structure intact via section and concept nodes and relationships between these.

Reading in the EAM textbook required reading in the PDF file and parsing it into its structural components, such as sections, paragraphs, or other elements, such as tables. These components are enriched with metadata, such as labels and textual content. The textual content is also converted into vector embeddings via an LLM. These vector embeddings enable similarity search

within the knowledge graph. Finally, all nodes and edges are persisted into the database. Nodes represent document fragments, chunks, sections, concepts, and embeddings.

With this method, the entire textbook can be queried via Cypher. The result is a knowledge graph in which nodes belonging to the same textbook section are connected within their own subgraph. A visualization of this is shown in figure 10.2.

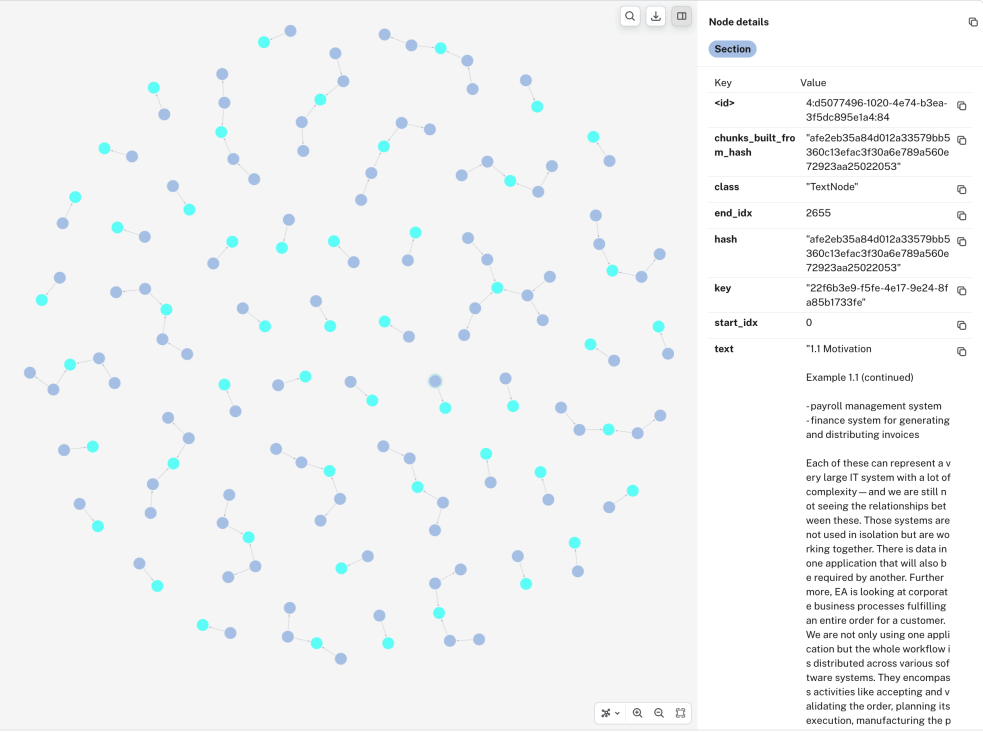

Figure 10.2: A screenshot of a subsection of the textbook's knowledge graph. It shows how different node types (e.g., documents and sections) are connected within their own sub-graphs. On the right-hand side is the metadata for a selected node. Notice the "text" label within the metadata where it is clear to see the actual contents of the textbook that derived this node.

The Jupyter Notebook that parsed the textbook was then iteratively refactored for each SpeedParcel file in accordance with its specific structure. The business capability support matrix was read in via an Excel file, while the business object model and the cross-application data-flow diagrams were already in XML format, which were exported directly from Archi. The approach for each of these custom parsers was the same: loading the respective local file, parsing their content into structures that align with their content (e.g., applications and capabilities), and adding them to the database.

This implementation method quickly reached its limits, as each custom parser was forked from the previous one, requiring manual adaptation to fit the new file. This would make parsing new files during a final review impossible without developing a proprietary parser for the user's new files. In parallel, the requirements for the final, real-world data were also becoming

clearer. This required building a generic, universal parser that better aligned with the new requirements. In the final prototype, only the XML parser remains, as the textbook and SpeedParcel parsers were only necessary for one-time parsing to fill the database with example data.

10.3.2 *Creating a Custom Knowledge Graph via the XML Parser*

The XML parser is what the prototype's end user uses to add their proprietary XML data to the knowledge graph.

XPath is a query language that operates on the tree structure of an XML document. This allows queries to be written for the XML content's elements, attributes, and text. It preserves the hierarchical structure of the content while doing so. [9] XPath is used within APOC's predefined procedure `apoc.load.xml()` to allow XML to be parsed and exposed for Cypher processing. [24] Code snippets in the appendix's section B.3.

The Node.js server orchestrates the XML parsing by taking the user's uploaded XML file and calling the Neo4j database with a Cypher containing the `apoc.load.xml()` procedure with XPath expressions inside of it. One Cypher for the structural elements and one Cypher for the relationships between these elements is called per uploaded XML file. Because the prototype requires reading XML files exported from Archi, the possible elements and relationships are known and finite. As a result, the Neo4j database performs the XML parsing internally via the APOC procedure.

If there is only a single child element of a given type, the corresponding map entry contains that element's value directly rather than a list. If there is more than one child element of the same type, the corresponding map entry contains a list of values, preserving the hierarchical XML structure. [24] This preserves the semantic structure of the original XML and enables subsequent Cypher queries to create nodes and relationships on the Neo4j server corresponding to this structure.

The APOC call uses the simplified parsing mode, meaning each child element type has its own entry in the parent map. This design choice simplifies Cypher traversal and can be changed by modifying the boolean value passed to the procedure call. [24]

The result of this is that the Node.js backend calls the Neo4j server with a Cypher containing the APOC procedure and XPath query embedded in the procedure. The Neo4j server executes this, adding the contents of the user's XML file to the knowledge graph. The XML parser was also implemented with the support of generative AI.

10.4 DATA FLOW

The sequence of interactions and how data flows between components are visualized in the sequence diagram in Figure 10.3. After the user enters their prompt in the frontend, it is forwarded to the backend. The backend then begins preprocessing the request by retrieving the current schema informa-

tion. This information is vital for the LLM in order for it to know how the knowledge graph is structured and how to create the Cyphers for it. Once the backend receives the schema information, the LLM converts the user's prompt into Cyphers using all the information that it has received via the context, schema, and prompt. The Cyphers are then used to query the selected knowledge graph stored in the Neo4j server. The Neo4j server returns raw data to the MCP server, which passes this to the backend. The backend then converts this raw response back into natural language using the LLM. Once converted, the backend saves the user's prompt and the system's response into the chat history before returning the response to the frontend, where it is displayed.

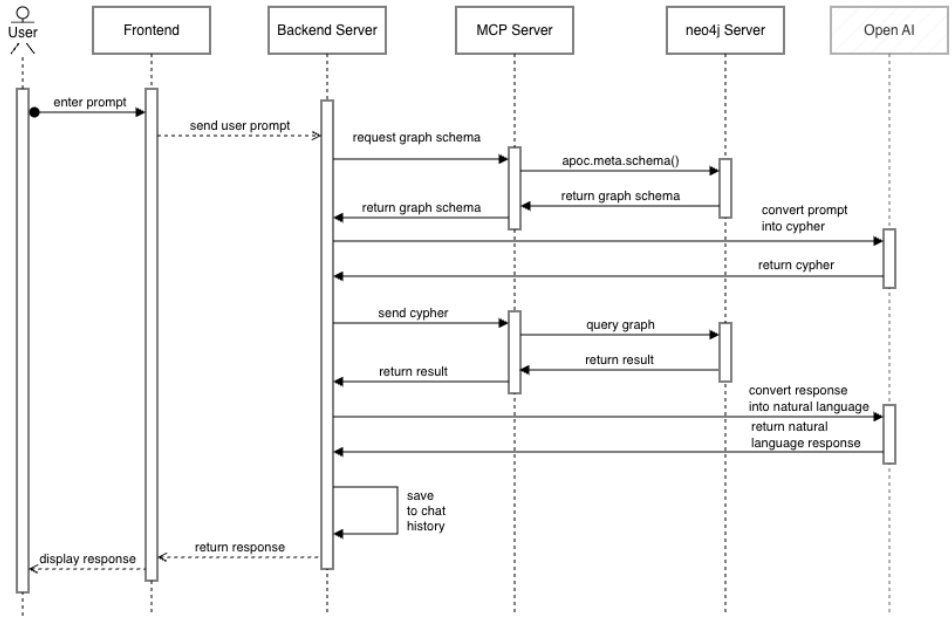

Figure 10.3: A UML sequence diagram showing the main flow of data when a user prompts the system.

This concludes the implementation for Masutā. All of the above-described components allow the system to take a user's natural language prompt and retrieve the relevant information from the databases. Together, these components make Masutā a cohesive, end-to-end prototype that demonstrates the technical feasibility of the proposed approach. Following this, chapter 11 discusses the results of this thesis.

11 RESULTS AND DISCUSSION

Now that it is understood how the study was conducted and what the architecture of the final prototype is, the results of the conducted Action Research and the state of the prototype can be categorically discussed. The results mainly reference the final review from chapter 9, but will also take a holistic look at the developed prototype.

11.1 KNOWLEDGE GENERATION AND TRANSPARENCY MECHANISMS

The first category addresses how Masutā generates its answers and how to achieve the best results when interacting with the system.

Differences in prompting style greatly influenced output characteristics. Ambiguous prompts, without domain-specific language and terms, left interpretation up to the LLM and often returned results that did not align with the expectations of the experts. However, when prompts were fine-tuned to resemble expert terminology and domain language, retrieval more frequently referenced elements in the knowledge graph containing the expected information. This is because the LLM has more context to build the Cypher with. For example, when supplementing the prompt with specific keywords, the LLM is able to generate a Cypher containing these keywords. The observed results exemplify how crucial the quality of prompting is for achieving a desired result.

Applied to real-world scenarios, this highlights two types of users who can benefit from this prompting strategy. First, it is an advantage if the user is aware of the details of the underlying knowledge graph, as this enables the user to prompt specifically for the knowledge graph at hand. If a user is aware that the knowledge graph was built from imported Archi models and understands the schema, they can add context to the prompt to retrieve the information they need to answer their question. A second user group who may achieve better results with the proprietary knowledge graph of the final review is users who understand the technical jargon underlying the implementation, in this example, the employees of the examination office. Even without the necessary technical know-how to prompt the system for keywords found in the knowledge graph, they are able to apply their own specialized terminology within the prompt. This specific context may help the LLM to generate the correct Cyphers to assist in such a use case. In contrast, a user who is neither aware of the underlying implementation nor knows the technical jargon of the examination office will struggle much more to achieve the desired result when interacting with the system.

This ties back to section 3.2, where Zhao et al. (2024) [45] describe that the context given to the LLM allows it to uncover hidden rationales, allowing it

to learn through the provided context. This implies that the user must also be active during interaction with the system and avoid naïve usage patterns if they are to extract the most value from it.

Next, the common challenge of LLMs hallucinating when generating answers could not be mitigated during this research. In the example of section 9.5, the observed reference to the QIS system illustrates this problem. Although QIS is not mentioned in the department's architecture, the LLM outputs and describes it as if it were, with no indication of uncertainty. This is a clear case in which the general knowledge on which an LLM is pretrained is leaked into the generated answer. Hallucinations such as this may go unnoticed, and although minuscule in the observed case, their effects in a real-world scenario could be large. The result of this is that the user must be aware that the LLM may hallucinate its answers, requiring them to constantly fact-check and ensure the correctness of outputs. When dealing with EAM data via a RAG-based system, it is necessary to separate graph-grounded knowledge from the model's trained knowledge.

Mitigating this risk would require additional steps beyond the current system design. For example, a more mature pipeline could incorporate a verification step in which generated answers are explicitly checked against the knowledge graph to ensure that they are grounded in it. Introducing stricter constraints in the query generation process, such as limiting the LLM to only use keywords found in the graph, may further reduce the likelihood of unsupported assumptions entering the response. However, these measures were not implemented within the scope of this work. In a real-world setting, addressing this challenge becomes essential, as uncontrolled leakage of pretrained knowledge into responses can undermine the reliability of the system.

This leads to the key advantage that Masutā has over commercial LLM tools such as Rovo, which is its transparency in how answers are generated. Masutā allows the user to copy and manually execute the exact Cypher query used to retrieve the data in Neo4j, making the underlying information directly inspectable. In contrast, Rovo does not provide insight into how its responses are derived. This inspectability enables users to validate outputs against the knowledge graph and detect potential hallucinations. The distinction therefore lies between a black-box system such as Rovo and a grey-box system such as Masutā, where the reasoning path remains transparent.

In a real-world setting, this added transparency could make a difference when discussing how to implement an LLM, particularly regarding data integrity and risk management. The enabled transparency allows for the data to be double-checked, ensuring that the data's integrity is upheld. At the same time, the risk of introducing misinformation into an enterprise's architecture can be reduced.

The challenge of dealing with hallucinations goes beyond the scope of Masutā, as Rovo also produced factually incorrect answers. This comparison between Masutā and Rovo highlights the need for features that enable this level of transparency. This is also confirmed in the paper by Zhao et al.

(2024), where the authors mention that "enhancing transparency in LLMs and reducing the uncertainty of the generated output is critical for increasing trust and reliability in their outputs, especially in fields where precision and accountability are paramount"[45] such as in the field of EAM.

The above points were a recurring theme throughout development and the final review. They show that having an understanding of the system at hand can give the user an advantage to extract the most value from it, as well as to mitigate errors.

11.2 ARCHITECTURAL LIMITATIONS

The final review identified several technical limitations in Masutā's current version.

A learning moment presented itself during the final review when Masutā was revealed to have tricked the Cypher generation process. The Cypher queries containing a templated answer showcase how relevant the underlying system prompt is. The prompt enables the LLM to complete the task, but does not ensure the correctness of the completed task. The predefined system prompt tasks the LLM to generate one Cypher per user request. It is supplemented with information on how to complete the task as well as some pointers on how to structure the Cypher. However, the system prompt never explicitly tells the LLM not to hardcode any self-generated information into the Cypher. This bypasses the knowledge extraction from the knowledge graph and essentially executes an empty Cypher. The result is that the LLM satisfies the given instruction by generating a Cypher and using the results of the Cypher to generate the final response. But it does not do so in a meaningful way by extracting relevant information from the knowledge graph. This highlights that the LLM may be following instructions, but it is not following the intent of the system prompt. The LLM itself may be optimized for instruction satisfaction but not task correctness.

Although this loophole in the system prompt was abused by the LLM, catching this behavior attests to the transparency advantage of Masutā, which allowed this loophole to be caught. Fixing the problem is as simple as adding a further instruction in the system prompt to prohibit this behavior. This highlights that the system's context serves as a governance layer, enabling the LLM to correctly fulfill the task assigned to it. A finely tuned system prompt is thus an important requirement for any system that outsources Cypher generation to an LLM.

In several interactions, Neo4j-related error messages were displayed. These errors were particularly associated with attempts by the LLM to write to the database. The current system architecture prevented permanent modifications to the database during the workshop session, as all Cyphers executed by the LLM were done in read-only mode. This design choice was deliberate, so as not to allow an uncontrolled modification of the knowledge graph by the LLM. From a research perspective, this restriction was necessary to guarantee data integrity during experimentation and to ensure reproducibil-

ity of results. In a more mature version of Masutā, write operations could be enabled through controlled interfaces, thereby expanding practical applicability.

A further technical limitation was identified while tasking the LLM to generate an Excel file. While OpenAI does support this file generation and output, the frontend and backend handling within Masutā does not currently implement the necessary mechanisms to receive, store, and transmit such artifacts. This means that even if the LLM successfully generated the file server-side, the system architecture lacked the capability to display the results to the user. The limitation is therefore not rooted in the LLM's reasoning capability, but in missing infrastructural components. It highlights that the prototype primarily focuses on query generation and knowledge retrieval rather than comprehensive document management functionality.

Even when no errors were being thrown, Masutā still displayed further technical shortcomings. One such moment was identified during the chain of prompts requesting the business support matrix. Even after multiple, refined prompts requesting a matrix, the system would not comply and instead responded with unrequested analysis. This case highlights that an LLM may not inherently understand the operational objective behind a request. To fulfill a concrete task, it must therefore be meaningfully guided by structured instructions and clearly defined expectations. While this level of orchestration was manageable during the academic review, it demonstrates that the current system requires deliberate steering to reliably produce task-oriented outputs in a real-world context.

This can be contrasted with Rovo's behavior, which did not throw any errors during the final review and demonstrated technical stability throughout the process. The system consistently produced coherent outputs without requiring manual intervention or refined steering. This level of robustness indicates greater technical maturity and integration readiness.

The above results demonstrate technical immaturity within Masutā. However, within the scope of this research, a certain degree of technical immaturity was expected and accepted. Masutā was conceived and evaluated as a proof of concept to validate architectural principles. It is not meant as a production-ready enterprise tool. Qualitative aspects such as stability represent a separate development trajectory from the core research objective of transparent knowledge graph interaction. Within that objective, functional validity was successfully demonstrated, even if technical robustness remains incomplete. These shortcomings would typically be addressed in a subsequent engineering phase and therefore constitute opportunities for future work rather than contradictions of the research objectives.

11.3 MAKE OR BUY - COMPARING MASUTĀ TO ROVO

The make-or-buy decision is a central consideration when introducing new applications within an enterprise architecture. It essentially asks whether a system should be developed in-house or procured as a standard solution

from the market. The default is typically to buy, as this is often cheaper and involves less risk. However, this decision is highly dependent on the business relevance of the underlying capability. Capabilities that are core to the business and directly contribute to competitive advantage may justify a custom-built solution, whereas common supporting functions are usually better served by mature, off-the-shelf applications. In practice, this decision is not black and white, but rather requires weighing multiple criteria against each other, including strategic importance, market availability, internal expertise, and overall cost-benefit. [17, pgs. 84-87]

Although the prototype is not meant as a mature system ready for real-world application, it serves well as a discussion point for a typical make-or-buy decision, asking whether licensing a RAG-based LLM tool or building a custom tool is more advantageous for an enterprise. The argument can be framed as buying a black-box system versus developing one's own grey-box system. The decision is therefore both technical and strategic.

A central advantage of a custom tool such as Masutā lies in its ability to incorporate expert knowledge via RAG. The knowledge graph containing the entirety of an EAM textbook allowed generated answers to be grounded in explicit domain knowledge. This grounding mechanism enables a system to expand its knowledge beyond the pretrained general knowledge of an LLM. A system such as Rovo cannot achieve this level of domain specificity without access to a proprietary knowledge graph, as it relies on the information of the pretrained model and the information available in Confluence. While Rovo can summarize and interpret the documented architecture information at hand, it cannot systematically embed structured expert knowledge in the same transparent and controllable manner as Masutā. This distinction becomes particularly relevant in specialized domains such as EAM, where correctness and conceptual alignment are critical.

Another decisive advantage of a self-developed system is extensibility. Because the codebase for Masutā is fully under internal control, it can be extended to fit any desired use case, given enough time and resources to implement such changes. The risk of extending Masutā is also rather low, as the codebase remains manageable in size, modular in structure, and fully under internal control, allowing new features or architectural adjustments to be implemented without dependency on an external provider. Through the MCP server, further systems can be connected in a standardized manner, allowing integration beyond Masutā's scope. In contrast, Rovo is constrained by Atlassian's functionality. Enterprises utilizing Rovo are dependent on the vendor's changes and cannot directly modify or extend the system's internal logic.

This extensibility directly relates to reducing the risk of vendor lock-in. When relying entirely on a third-party system such as Rovo, an enterprise becomes dependent on external decisions regarding pricing models, features, and product strategy. A custom-built solution such as Masutā mitigates this dependency by preserving architectural control. The enterprise

retains control over its data, interfaces, and system evolution, with the exception of the underlying LLM provider.

However, the final review clearly demonstrated that Rovo currently represents the more mature solution. The system generated responses in a consistent, structured format, often restating the question and summarizing relevant context before providing an answer. This behavior was noted favorably by the experts, as it indicated a correct interpretation of the task and ensured clarity. Rovo operated without visible system errors and required less interaction to steer. In contrast, Masutā required more deliberate prompting and exposed instabilities that would not be acceptable in a production environment. From a usability and stability perspective, Rovo therefore demonstrated a higher degree of operational maturity.

At the same time, this maturity comes at the cost of transparency. Rovo does not allow inspection of how information is retrieved or how conclusions are derived. In cases of incorrect or misleading outputs, the user has no means to retrace the reasoning path. While Rovo was not immune to hallucinations, its black-box nature limits the ability to detect and analyze them. In contrast, Masutā exposes the generated Cypher queries and the underlying raw graph data, as previously discussed, allowing answers to be verified against their source. This transparency shifts responsibility to the user by enabling traceability, which may be a critical point in enterprise contexts.

Therefore, the make-or-buy decision becomes a trade-off between maturity versus control. Rovo represents a stable, polished, and immediately deployable solution. Masutā represents a flexible, transparent, and extensible architecture that would require further engineering effort to reach comparable stability. Enterprises must therefore decide whether they prioritize convenience and abstraction, or architectural control and transparency.

11.4 PRACTICAL APPLICABILITY IN EAM

Masutā has implications for how enterprise architects in general interact with architectural data. Rather than manually using modeling tools, the enterprise architect can formulate a task in natural language and receive a structured response grounded in architectural data. This reduces friction when information is present in the architecture repository but is difficult to find or extract. During the workshop, experts used the generated outputs as a basis for describing and reflecting on the current state of the landscape. This indicates that the system does not merely retrieve data, but can actively support architectural analysis.

Looking at the work of a junior level enterprise architect, this enables them to deal with data that may otherwise be out of scope for their skill level. With the support of such a system, a junior architect may be able to gain an understanding of the data and develop a course of action for the architecture that would otherwise not be possible as a junior.

The discussion during the development also raised the question of whether a graph database is strictly necessary for such a system. It was argued that

in smaller scopes, an LLM may be able to construct contextual relationships in a more implicit, black-box manner without a formalized graph structure. However, once a certain level of scale and complexity is reached, explicit structuring of relationships becomes essential. In such cases, the graph database does not only serve as a knowledge-base, but as a transparent and inspectable foundation for reasoning. The combination of a knowledge graph and an LLM therefore becomes particularly relevant in enterprise contexts, where traceability, scalability, and avoidance of vendor lock-in are decisive factors.

11.5 LEARNINGS FROM THE IMPLEMENTATION

Many learnings can be extracted from the implementation of Masutā. This process revealed that the implementation of a RAG-based system should not be overly reliant on LLMs. In early cycles, several core functions for transforming the data were delegated to the LLM. While this accelerated prototyping, it introduced instability and unpredictability in core system components. Replacing LLM-based parsing with deterministic APOC procedures significantly improved reproducibility and trust in the resulting knowledge graph. This shows that LLMs are powerful for reasoning and language generation, but it should be considered to implement critical infrastructure components with deterministic mechanisms.

A further learning from the development phase concerns schema awareness. Pre-querying the database for its schema so that Cyphers can be generated dynamically proved to be a major architectural improvement. Hard-coding the schema into the system prompt was sufficient as long as the dataset remained stable. However, once heterogeneous datasets were introduced, such as when users were able to import their own XML files, this approach no longer scaled well. The shift to retrieving the current schema via `apoc.meta.schema()` and injecting it into the system context enabled the prototype to reach a level where it could successfully answer all three categories of questions. This allowed the LLM to operate on an up-to-date representation of the graph structure at runtime. This also demonstrates that text-to-Cypher generation is only robust when the LLM has access to a representation of the knowledge graph. Schema retrieval is therefore a core feature within Masutā.

A final key learning concerns the importance of the system prompt as a control mechanism for the LLM. Throughout the development cycles and especially during the final review, it became evident that the system prompt effectively defines the behavioral guardrails of the LLM. The observed Cypher trick was not a failure of the LLM, but a consequence of insufficiently constrained instructions. This demonstrates that the system prompt is not merely contextual guidance, but also a control layer that allows a model to better interpret the task at hand. Therefore, designing, refining, and testing the system prompt must be considered a critical activity.

In summary, the implementation demonstrated that building a transparent, schema-aware, and extensible LLM system requires a deliberate architecture to enable this. LLM capability alone is not sufficient to guarantee a fully functional prototype. Only through deterministic ingestion, dynamic schema awareness, and controlled query generation can a system such as Masutā operate in a way that allows users to interact with the knowledge graph via natural language while remaining both inspectable and adaptable to enterprise environments.

11.6 OVERALL GOAL FULFILLMENT

With the conclusion of the final review and the preceding discussion of the results, a retrospective comparison between the expected outcome from the beginning of the research and the current state of Masutā is appropriate.

Before the first development cycle commenced, it was unclear what the final system was to look like. From vague definitions in the beginning of a chatbot that allows enterprise architects to interact with their data via natural language, it was unclear how this goal would be achieved. With each iterative development step during the conducted Action Research, the end goal became increasingly clear. The result of this is the final version of the prototype.

The research's architectural objectives can be considered fulfilled. An end-to-end pipeline was successfully implemented, enabling the ingestion of architectural data, its representation within a knowledge graph, the generation of Cypher queries from natural language prompts, and the transparent retrieval of results. The ability to inspect generated Cyphers and validate the underlying data directly in Neo4j demonstrates that the intended grey-box design was achieved.

At the same time, the final review revealed clear limitations in terms of maturity. The system requires structured prompting, is susceptible to hallucinations, and does not yet provide the robustness expected from a production-ready enterprise solution. These shortcomings, however, do not contradict the research contribution. Rather, they highlight that architectural feasibility and operational stability are separate dimensions. The prototype validates the expected concept, even if further engineering effort would be required for a more mature deployment.

Overall, Masutā fulfills its primary objective as a proof of concept artifact within an Action Research setting. It demonstrates that natural language interaction with a proprietary EAM knowledge graph is technically feasible and enables transparent, extensible interaction mechanisms. The remaining gaps lie not in conceptual viability, but in the path from prototype to mature enterprise system. Therefore, the core goal of this research, whether a transparent, RAG-based interaction mechanism for EAM data can be realized, is answered positively.

In summary, this chapter has shown that the prototype succeeds architecturally while remaining operationally immature. It highlights both the

potential and the structural constraints of integrating LLMs with enterprise knowledge graphs. The findings demonstrate that transparency, schema awareness, and deterministic infrastructure are decisive design principles for developing a RAG-based chatbot system in the EAM domain. These results form the foundation for the concluding reflections and future research directions discussed in the next chapter.

12 CONCLUSION AND FUTURE WORK

Enterprise Architecture Management requires the analysis and maintenance of large amounts of interconnected architectural information, which is often distributed across heterogeneous documentation formats and tools. Retrieving and editing the relevant information is a challenge of its own for enterprise architects, as this must be done manually and requires a certain level of experience. AI chatbots can support this process by allowing the enterprise architect to interact with the data through natural language. However, such chatbots are typically black-box systems and introduce problems of their own, which may hinder their adoption in real-world settings.

The objective of this research was to investigate whether a transparent RAG–based chatbot can support enterprise architects in accessing proprietary architectural knowledge through natural language interaction. The result is the finished prototype Masutā. Using an LLM to generate queries based on the user's natural language input, it is able to retrieve relevant data from a proprietary knowledge graph in order to supplement its responses. Masutā was created through Action Research, in which the researcher, along with two practitioners, iteratively developed the system across four cycles. Throughout each cycle, the current problems were diagnosed, action plans were made, and actions were taken before evaluating and specifying the learnings from the cycle. This enabled a collaborative, exploratory approach.

The results of the conducted Action Research cycles and the final evaluation demonstrate that natural language interaction with a proprietary EAM knowledge graph is technically feasible. Masutā demonstrates this through its end-to-end pipeline, allowing a user to upload proprietary enterprise architecture data into the knowledge graph and subsequently querying it. The retrieval process is done by transforming the user's input into a Cypher query using an LLM. This query is then executed within the Neo4j database. The LLM's natural-language response to the user is then augmented with information retrieved from the database. Along with the response, the user is also returned the exact Cypher used, allowing them to manually execute it within Neo4j. This enables the user to inspect the results and ensure factual correctness of the LLM's response.

One of the central insights of this research is the importance of transparency mechanisms when integrating large language models with enterprise knowledge systems. A central problem when working with LLMs is their hallucinated responses. By showing which Cypher is used to generate the response, Masutā is able to mitigate its hallucinations, as a user is able to validate the factual correctness of the generated response. At the same time, the final review showed that the quality of generated answers depends strongly on the prompt context and the specialist terminology used by the

user. Without sufficient context, the system does not reliably infer the correct link between the user's prompt and the corresponding part of the knowledge graph. This further highlighted that schema awareness and a well-designed system prompt are necessary in order for the LLM to generate meaningful Cyphers for a heterogeneous knowledge graph. Another central insight is that critical system functions, such as data ingestion and schema handling, should not be left entirely up to the LLM but require deterministic mechanisms in order to achieve robustness and reproducibility. In this way, the research showed that the value of such a system lies not only in natural language interaction with enterprise architecture data, but in doing so in a way that remains inspectable, controllable, and extensible. These insights emerged throughout the development and final review, justifying the applied Action Research method.

12.1 FUTURE WORK

Despite the promising results, the developed prototype remains a proof of concept artifact rather than a production-ready enterprise solution. The following discusses possible improvements and future work on Masutā.

Several technical improvements could further increase Masutā's robustness and usability. For example, the system is very sensitive to its prompt context. Ambiguous or loosely formulated prompts can lead to the generation of Cypher queries that do not target the correct node types within the knowledge graph. Improving the schema-awareness of the prompting strategy or introducing intermediate validation steps could help reduce this dependency on prompt formulation.

Another limitation concerns the use of the database schema within the prompt context. Currently, the schema is supplied before each prompt in order to guide the LLM in generating valid Cypher queries. While this approach improves correctness, it may become problematic as the knowledge graph grows. Prior research has shown that providing extensive schema descriptions can overload the LLM's context window and negatively impact performance. [21, 38] Future work could therefore investigate more efficient methods for supplying schema information.

Furthermore, the data parsing used for transforming Archi XML exports into the knowledge graph may add unnecessary information to the graph. For example, the exported XML files from Archi contain not only the architectural elements themselves but also additional metadata related to visualization and layout. Currently, this information is not filtered out during parsing. Investigating whether this data introduces noise into the knowledge graph and refining the parsing accordingly could further improve retrieval quality.

Beyond improving the technical robustness of the prototype, future work should also address the requirements for deploying Masutā in real-world enterprise environments.

A major requirement in such environments is the management of access rights to sensitive information stored in the knowledge graph. This compliance requirement falls into the responsibilities of an enterprise architect. In practice, not every user should be allowed to view all architectural elements or related documentation. Future work should therefore investigate how role-based access control can be integrated into the interaction pipeline, ensuring that generated Cypher queries and returned answers are constrained by the permissions of the respective user.

Another relevant improvement concerns the handling of artifacts generated from system responses. During the final review, it became apparent that downloadable files generated from OpenAI responses, such as Excel files, would enhance the system's practical usefulness. Extending the current frontend and backend so that generated files can be received, stored, and made available for download would therefore be a meaningful step toward a more complete application workflow.

In addition, the current interaction flow could be extended by replacing the largely direct prompt-to-query process with a more explicit pipeline of intermediate steps. Rather than relying on a single generation step to interpret and fulfill the prompt, future work could investigate orchestration-based approaches in which the question is first analyzed, broken down into smaller subtasks, and processed step by step. Such a pipelined interaction strategy may improve robustness and make the system's internal processing more controllable.

Because Masutā already uses MCP as a standardized interface to external systems, future work could also explore connecting it to additional MCP-enabled applications. For example, instead of limiting the system to retrieving and explaining architectural information, Masutā could be extended to interact directly with tools such as Archi or Confluence. This would allow the system not only to analyze architectural data but also to support follow-up tasks, such as updating documentation. Such extensions would move the system further from a question-answering prototype toward a broader architecture support system.

Overall, this research demonstrates that transparent interaction with enterprise architecture knowledge through a RAG-based system is technically feasible. By combining large language models with a structured knowledge graph and exposing the generated queries used for retrieval, Masutā provides a grey-box interaction mechanism that reduces the opacity typically associated with AI-based systems. While the prototype represents only an initial step toward practical deployment, the results indicate that a transparent, schema-aware chatbot system can play a meaningful role in supporting enterprise architects as they navigate complex architectural landscapes.

BIBLIOGRAPHY

[1] Shoaib Ahmed, Nazim Taskin, David J Pauleen, Jane Parker, et al. "Motivating information technology professionals: The case of New Zealand." In: *Australasian Journal of Information Systems* 21 (2017).

[2] Anthropic PBC. *Introducing the Model Context Protocol*. 2025. URL: https://www.anthropic.com/news/model-context-protocol (visited on 01/02/2026).

[3] Richard L Baskerville. "Investigating information systems with action research." In: *Communications of the association for information systems* 2.1 (1999), p. 19.

[4] Gavin Bierman, Martín Abadi, and Mads Torgersen. "Understanding typescript." In: *European Conference on Object-Oriented Programming*. Springer. 2014, pp. 257–281.

[5] Jan vom Brocke, Alexander Simons, Bjoern Niehaves, Björn Niehaves, Kai Riemer, Ralf Plattfaut, and Anne Cleven. "Reconstructing the giant: On the importance of rigour in documenting the literature search process." In: (2009).

[6] Robert Buchmann, Johann Eder, Hans-Georg Fill, Ulrich Frank, Dimitris Karagiannis, Emanuele Laurenzi, John Mylopoulos, Dimitris Plexousakis, and Maribel Yasmina Santos. "Large language models: Expectations for semantics-driven systems engineering." In: *Data & Knowledge Engineering* 152 (2024), p. 102324.

[7] Chakra Systems. *Chakra UI is a component system for building products*. https://chakra-ui.com/. Online; accessed 2026. 2026.

[8] Jialin Chen, Houyu Zhang, Seongjun Yun, Alejandro Mottini, Rex Ying, Xiang Song, Vassilis N Ioannidis, Zheng Li, Qingjun Cui, and Yale University Amazon. "GRIL: Knowledge Graph Retrieval-Integrated Learning with Large Language Models." In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. 2025, pp. 16306–16319.

[9] James Clark, Steve DeRose, et al. *XML path language (XPath)*. 1999.

[10] Flora Cornish, Nancy Breton, Ulises Moreno-Tabarez, Jenna Delgado, Mohi Rua, Ama de Graft Aikins, and Darrin Hodgetts. "Participatory action research." In: *Nature Reviews Methods Primers* 3.1 (2023), p. 34.

[11] Dragon1 Inc. *The Business Case for Dragon1*. https://www.dragon1.com/downloads/dragon1-the-business-case.pdf. Accessed: 2026-03-27. 2017.

[12] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. 2025. arXiv: 2404.16130 [cs.CL]. URL: https://arxiv.org/abs/2404.16130.

[13] Muhammad Usman Hadi, Rizwan Qureshi, Abbas Shah, Muhammad Irfan, Anas Zafar, Muhammad Bilal Shaikh, Naveed Akhtar, Jia Wu, Seyedali Mirjalili, et al. "Large language models: a comprehensive survey of its applications, challenges, limitations, and future prospects." In: *Authorea preprints* 1.3 (2023), pp. 1–26.

[14] Michael Townsen Hicks, James Humphries, and Joe Slater. "ChatGPT is bullshit." In: *Ethics and Information Technology* 26.2 (2024), pp. 1–10.

[15] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. "Model context protocol (mcp): Landscape, security threats, and future research directions." In: *arXiv preprint arXiv:2503.23278* (2025).

[16] Jürgen Jung. "Purpose of enterprise architecture management: investigating tangible benefits in the german logistics industry." In: *2019 IEEE 23rd International Enterprise Distributed Object Computing Workshop (EDOCW)*. IEEE. 2019, pp. 25–31.

[17] Jürgen Jung and Bardo Fraunholz. *Masterclass Enterprise Architecture Management*. Springer, 2021.

[18] Emanuele Laurenzi, Adrian Mathys, and Andreas Martin. "An LLM-aided enterprise knowledge graph (EKG) engineering process." In: *Proceedings of the AAAI Symposium Series*. Vol. 3. 1. 2024, pp. 148–156.

[19] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. "Retrieval-augmented generation for knowledge-intensive nlp tasks." In: *Advances in neural information processing systems* 33 (2020), pp. 9459–9474.

[20] LlamaIndex. *Parsing PDFs with LlamaParse: a how-to guide*. 2025. URL: https://www.llamaindex.ai/blog/pdf-parsing-llamaparse (visited on 01/02/2026).

[21] Nandana Mihindukulasooriya, Sanju Tiwari, Carlos F Enguix, and Kusum Lata. "Text2kgbench: A benchmark for ontology-driven knowledge graph generation from text." In: *International semantic web conference*. Springer. 2023, pp. 247–265.

[22] Vinaytosh Mishra and Monu Pandey Mishra. "PRISMA for review of management literature–method, merits, and limitations–an academic review." In: *Advancing methodologies of conducting literature review in management domain* (2023), pp. 125–136.

[23] Neo4j, Inc. *Indexes*. Cypher manual documentation on index definition and usage in Neo4j. 2025. URL: https://neo4j.com/docs/cypher-manual/current/indexes/ (visited on 01/01/2026).

[24] Neo4j, Inc. *Load XML*. Technical documentation for the apoc.load.xml procedure in Neo4j. 2025. URL: https://neo4j.com/labs/apoc/4.2/import/xml/ (visited on 01/01/2026).

[25] Neo4j, Inc. *Neo4j MCP Server*. GitHub repository. 2025. URL: https://github.com/neo4j/mcp (visited on 01/02/2026).

[26] Neo4j, Inc. *apoc.meta.schema*. Technical documentation for the APOC library in Neo4j. 2025. URL: https://neo4j.com/labs/apoc/4.1/overview/apoc.meta/apoc.meta.schema/ (visited on 01/01/2026).

[27] Neo4j, Inc. *Neo4j Graph Database*. https://neo4j.com/. Accessed 2026. 2026.

[28] OpenAI. *ChatGPT (January 06 version)*. https://chat.openai.com/chat. [Large language model]. 2023.

[29] OpenAI. *OpenAI API Documentation*. Technical documentation for accessing large language models via the OpenAI API. 2025. URL: https://platform.openai.com/docs/overview (visited on 01/01/2026).

[30] David Osei Opoku, Ming Sheng, and Yong Zhang. "Do-rag: A domain-specific qa framework using knowledge graph-enhanced retrieval-augmented generation." In: *arXiv preprint arXiv:2505.17058* (2025).

[31] Matthew J Page, Joanne E McKenzie, Patrick M Bossuyt, Isabelle Boutron, Tammy C Hoffmann, Cynthia D Mulrow, Larissa Shamseer, Jennifer M Tetzlaff, Elie A Akl, Sue E Brennan, et al. "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews." In: *bmj* 372 (2021).

[32] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph databases: new opportunities for connected data*. "O'Reilly Media, Inc.", 2015.

[33] Rainer Schlör and Jürgen Jung. "Analysis using the business support matrix: elaborating potential for improving application landscapes in logistics." In: *2018 IEEE 22nd International Enterprise Distributed Object Computing Workshop (EDOCW)*. IEEE. 2018, pp. 192–199.

[34] Aditya Sharma, Luis Lara, Christopher J Pal, and Amal Zouaq. "Reducing hallucinations in language model-based sparql query generation using post-generation memory retrieval." In: *arXiv preprint arXiv:2502.13369* (2025).

[35] Karan Singhal, Shekoofeh Azizi, Tao Tu, S Sara Mahdavi, Jason Wei, Hyung Won Chung, Nathan Scales, Ajay Tanwani, Heather Cole-Lewis, Stephen Pfohl, et al. "Large language models encode clinical knowledge." In: *Nature* 620.7972 (2023), pp. 172–180.

[36] Muhamed Smajevic and Dominik Bork. "From conceptual models to knowledge graphs: a generic model transformation platform." In: *2021 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE. 2021, pp. 610–614.

[37] Rik Van Bruggen. *Learning Neo4j*. Packt Publishing Ltd, 2014.

[38] Yuwei Wan, Zheyuan Chen, Ying Liu, Chong Chen, and Michael Packianather. "Prompting large language models based on semantic schema for text-to-Cypher transformation towards domain Q&A." In: *Decision Support Systems* (2025), p. 114553.

[39] Quan Wang, Zhendong Mao, Bin Wang, and Li Guo. "Knowledge graph embedding: A survey of approaches and applications." In: *IEEE transactions on knowledge and data engineering* 29.12 (2017), pp. 2724–2743.

[40] Jane Webster and Richard T Watson. "Analyzing the past to prepare for the future: Writing a literature review." In: *MIS quarterly* (2002), pp. xiii–xxiii.

[41] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. "An explanation of in-context learning as implicit bayesian inference." In: *arXiv preprint arXiv:2111.02080* (2021).

[42] Joshua Yu. *graph-rag*. GitHub repository. 2025. URL: https://github.com/Joshua-Yu/graph-rag (visited on 01/02/2026).

[43] Stefano Zeppieri. "MMAG: Mixed Memory-Augmented Generation for Large Language Models Applications." In: *arXiv preprint arXiv:2512.01710* (2025).

[44] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. "Retrieval-augmented generation for ai-generated content: A survey." In: *Data Science and Engineering* (2026), pp. 1–29.

[45] Siyun Zhao, Yuqing Yang, Zilong Wang, Zhiyuan He, Luna K Qiu, and Lili Qiu. "Retrieval augmented generation (rag) and beyond: A comprehensive survey on how to make your llms use external data more wisely." In: *arXiv preprint arXiv:2409.14924* (2024).

Part II

APPENDIX

A INSTRUCTION MANUAL FOR MASUTĀ

Masutā is the name of the prototype enterprise architecture chatbot. Follow these instructions to download Masutā, set it up, and learn how to use it.

When executing the commands from the terminal, always execute them from the root folder of the project. The best results are achieved by executing each command individually (i.e. copy-paste the commands individually line-for-line instead of the entire block).

A.1 DOWNLOADING MASUTĀ

In order to get Masutā up and running, a few prerequisites will have to be fulfilled.

The first is to download the application code for Masutā from GitHub: https://github.com/HendrikDA/masuta-ea-chatbot-prototype. Download the entire folder as a .zip file (under Code→Download ZIP) and unpack it locally.

The second prerequisite is to download Docker and Docker Compose. Instructions on how to set this up on various operating systems can be found under this link: https://docs.docker.com/compose/install/.

An optional prerequisite is to download Neo4j Desktop. This allows the user to inspect the graph database within an interactive desktop application. Neo4j Desktop can be found here: https://neo4j.com/download/. However, this is not mandatory and the databases may also be inspected via neo4j's web-application.

A.2 FIRST TIME SETUP

A.2.1 *Setting Environment Variables*

Before being able to run any containers, a few environment variables must be set. This requires creating and editing `.env` files throughout the project. It may be required to use a code editor to execute this step. All `.env` files are prepared via the adjacent `.env.example` file of the individual folder. The following steps explain how to create the required `.env` files per sub-folder:

- **Project Root Folder**: Within the root folder of the project, simply duplicate the `.env.example` file and rename the duplicated file to `.env`. Ensure it is in the root folder of the project. No values in the `.env` file need to be changed for the prototype to work.

- **Frontend**: Within the frontend folder, simply duplicate the `.env.example` file and rename the duplicated file to `.env`. Ensure it is in the root

folder of the ./frontend/ folder. No values in the `.env` file need to be changed for the prototype to work.

- **MCP Backend**: Within the mcp-backend folder of the project, simply duplicate the `.env.example` file and rename the duplicated file to `.env`. Ensure it is in the root folder of the ./mcp-backend/ folder. Within the `.env` file, the value for **OPENAI_API_KEY** must be set. It may require that an API key is generated within OpenAI's API platform. All other values may remain as they are for the prototype.

Once these environment variables are set, the databases may be set up.

A.2.2 *Preparing the Databases*

Before being able to set up the databases with the example data, the backup files need to be downloaded and placed into the correct folders. They can be found under the following links:

- Textbook Data: neo4j-2025-11-16T12-53-54-fde218db.backup
- SpeedParcel Data: neo4j-2025-12-10T21-44-58-fde218db.backup

If the above files are not accessible, please reach out to the author or advisors of this thesis.

Once both files are downloaded they are ready to be placed within the project's folders. The Textbok Data backup file must be placed within the folder `./textbook-data/`. The SpeedParcel Data backup file must be placed under `./speedparcel-data/`. This ensures that the backup files are present and in the correct folders when running the Docker commands to set up the databases in the next step.

A.2.3 *Setting Up the Databases*

If Masutā is being run for the first time, then two containers need to be executed once so that they populate both databases with the default data. From within the root folder of the project, execute the following commands

```
docker compose --profile speedparcel-restore run --rm speedparcel-restore
docker compose --profile textbook-restore  run --rm textbook-restore
```

Windows: If the executing system runs on Windows, this step requires the use of WSL (Windows Subsystem for Linux) and to set the HOME variable via `SET HOME=\C:\Users\<username>`. It may be required to execute these commands from a normal terminal instead of the Windows PowerShell.

Once these commands have run through successfully, both databases are populated with the default data and are ready to be used.

A.2.4 *Hard-Resetting Everything*

If at any time a hard-reset for the entire application is required, run the following commands:

```
docker compose down
rm -rf $HOME/neo4j/data
rm -rf $HOME/neo4j_empty/data
```

After resetting the application, restart with the first time setup.

A.3 RUNNING THE APPLICATION

Running Masutā is as simple as running the following command:

```
docker compose up
```

When running this command for the first time, it may take a while as it will have to download dependencies.

With this command, four containers are started.

- The SpeedParcel Neo4j database filled with example data and the textbook
- The playground Neo4j database filled with only the textbook
- The MCP-Backend which is the backend node.js which by default runs under http://localhost:4000
- The frontend which by default runs under http://localhost:3000/

If Docker does not automatically open the frontend in the browser, navigate to it under http://localhost:3000/. Everything is now ready and Masutā may be used.

A.4 FEATURE OVERVIEW

This section details the features within Masutā.

A.4.1 *Input Field and Chat History*

At the bottom of the application is an input field where the user may prompt Masutā. Pressing enter sends the prompt. The user's input prompt then appears as a chat bubble bound to the right of the chat-area. The system then starts working on answering the prompt and displays a chat bubble bound to the left of the chat-area displaying "Thinking...". As soon as the application is done thinking, the response is displayed in the same chat bubble.

Masutā's response has a button appended to it which allows the user to copy the Cypher which was used to generate the answer. The user may take this copied Cypher and paste it into Neo4j desktop to inspect the raw response that was used to generate the answer.

A.4.2 *Toggling the Database*

At the top right of the application is a toggle switch. This allows the user to toggle between the SpeedParcel database and the playground database.

A.4.3 *Resetting the Playground Database*

Under the menu at the top left, the user is presented with the option to reset the database. This option is only available if the playground database is selected via the toggle switch. After confirming the reset in the dialog that pops up, the playground is reset to its default state with only the textbook information. The SpeedParcel database cannot be reset via the UI.

A.4.4 *Importing Custom Data*

Under the menu at the top left, the user is presented with the option to import their own data. This option is only available if the playground database is selected via the toggle switch. Within the dialog that pops up, the user can add up to 10 files either by selecting them via the file system or by dragging-and-dropping them into the dialog. Only .xml files are accepted. Check section A.5 on how to export XML files from ArchiMate.

After clicking the import button, the files are uploaded. If the upload was successful, an alert is shown notifying the user of this. The uploaded data remains within the realms of the Docker containers and is not uploaded to any third-party sources. The data can then be queried if the database toggle is set to the playground database.

A.4.5 *Inspect Database*

Under the menu at the top left, the user is presented with the option to view the graph data in Neo4j browser their own data. This option is available for both the SpeedParcel and playground database. Clicking this opens another tab under https://console-preview.neo4j.io/tools/query. Here, the user can connect to the selected database. The connection string is displayed in a dialog within Masutā. Here, the user can inspect the graph structure and paste the Cyphers used for the generated responses.

In order to access the Neo4j console, it may be required that the user has a Neo4j account.

A.4.6 *Chat Context and Building on Previous Answers*

Masutā saves the chat history so that the user is able to build upon previous prompts. The chat history is set so that the single previous user prompt and corresponding response is saved. This means that a further question can be

asked about the previous prompt or its response. The chat history's context only goes back one prompt and response.

A.4.7 *Exported Chat Protocol*

In order to support the explorative nature of the prototype, a built-in function that runs in the background of Masutā logs all interactions between the user and the system. This protocol can be found in the project's directory under `./mcp-backend/protocols/chat_history.txt`. The log's schema is always the same and looks as follows:

```
--- New Chat Turn <current date as YYYY-MM-DDTHH:mm:ss.SSSZ> ---
Using DB Target: <SpeedParcel or Playground>
User: <Prompt>
Agent: <Response>
Executed Cypher: <Cypher>
```

Listing A.1: Schema of the automatically exported protocol

Each turn between the user and the chatbot is appended to the end of the text file. The chat history is never reset and is thus persistent across multiple sessions. This protocol can be helpful in documenting results when interacting with Masutā.

A.5 EXPORTING XML FILES FROM ARCHIMATE

It is recommended to import XML files into Masutā that have been exported directly from Archi. To do so, within the Archi application, click on file → export → model to open exchange file. Save this to a location you can find later. Notice that the exported file is an XML file. This can then be imported into Masutā as described in section A.4.4.

A.6 USING A DIFFERENT LLM

Although Masutā was implemented with the intention of using an LLM from OpenAI, it is technically possible to use a different LLM. To do so, the source code needs to be edited in x different locations as follows:

- ./mcp-backend/.env: Set the variable of the OPENAI_API_KEY to the API key of whatever new LLM you wish to use.
- ./mcp-backend/src/server.ts: The root server file needs to have all calls toward OpenAI replaced with calls to the new LLM. These can be found in the functions `nlToCypher()` and `explainResult()`. The structure of the calls made and the way they are subsequently handled may have to be refactored as well. It must also be ensured that the structure of the response to the frontend remains the same, otherwise a refactoring on the frontend will also be necessary.

A.7 EXAMPLE QUESTIONS

Questions can be broken down into three categories:

1. Category 1: Textbook questions about enterprise architecture management
2. Category 2: Basic information about the enterprise architecture data
3. Category 3: Questions that span both category 1 and 2 and are more complex than questions in category 2. These require more in-depth information about the data as well as how to deal with it as an enterprise architect.

The following list of questions serve as examples that Masutā can handle. These simply serve as ideas to test the system - the user may input their own thought up questions.

1. Questions in Category 1
   - What are the schools of enterprise architecture management?
   - How does enterprise architecture relate to town planning?
2. Questions in Category 2
   - How many capabilities are supported?
   - How many applications are in use?
   - Which application supports the most capabilities?
   - If I remove application x, which capabilities will be affected?
3. Questions in Category 3
   - We plan on removing application x. What do we have to look out for when doing so?
   - We wish to implement a new application x which covers the capabilities x, y, and z. Which existing applications may become redundant?
   - Which capabilities currently rely on single-point-of-failure applications? How should we deal with this?
   - Which capabilities are over-supported by multiple applications?
   - What interfaces does application x have?
   - How big of a task is it to migrate from application x to application y?

This concludes the instruction manual for Masutā. Beyond the example questions, users are encouraged to prompt Masutā to explore the example data provided in SpeedParcel or to analyze their own proprietary data after importing it, in order to gain insights into their enterprise architecture.

B

MASUTĀ CODE SNIPPETS

This chapter showcases code snippets which are of special interest as they show how key functionalities of Masutā work. The code snippets do not represent the full application and may be incomplete. Please reference the GitHub project for the full codebase under https://github.com/HendrikDA/masuta-ea-chatbot-prototype.

B.1 DYNAMIC SCHEMA EXTRACTION AND INJECTION FOR QUERY GENERATION

This snippet shows how the system dynamically retrieves the current structure of the Neo4j knowledge graph using `apoc.meta.schema()` and available indexes, summarizes it into a textual representation, and injects it into the LLM prompt. By providing the schema at runtime, the LLM is guided to generate structurally valid and context-aware Cypher queries, reducing incorrect assumptions about labels, properties, and relationships.

```
// Append last turn to chat history for context
const promptWithContext = lastTurn
  ? "Previous conversation turn:
    User: ${lastTurn.user}
         Agent: ${lastTurn.agent}
  New user question: ${prompt}"
  : prompt;

// Fetch schema dynamically from MCP server
const apocSchemaRaw = await readCypher("CALL apoc.meta.schema();");
const indexesRaw = await readCypher("SHOW INDEXES;");
const schemaText = summarizeGraphSchema(apocSchemaRaw, indexesRaw);

// Convert natural language into Cypher
let cypher = await nlToCypher(promptWithContext, schemaText);
```

B.2 SCHEMA-AWARE TEXT-TO-CYPHER TRANSLATION VIA PROMPT ENGINEERING

This snippet defines the system prompt used to translate natural language questions into executable Cypher queries. It enforces strict constraints such as schema adherence, correct Neo4j syntax, and index utilization, while restricting the output to a single valid query. The prompt operationalizes schema-aware query generation by combining user input with dynamically retrieved schema information.

```
async function nlToCypher(nlPrompt: string, schema: string) {
const systemPrompt = "
```

```
  You are working with a Neo4j graph whose structure is described in the schema
      summary.

  Important rules:
  - Do not assume fixed labels like :Application or :Chunk unless they appear in
       the schema.
  - Infer meaning from label names (e.g. ApplicationComponent     application).
  - If no text-centric nodes exist, answer using structural relationships.

  Index usage based on the schema information passed to you:
  - If the schema summary lists a VECTOR index relevant to the task, start with:
        CALL db.index.vector.queryNodes("<indexName>", $embedding, <k>) YIELD
      node, score
  - If the schema summary lists a FULLTEXT index relevant to the task, start
      with: CALL db.index.fulltext.queryNodes("<indexName>", $query, {limit: <k
      >}) YIELD node, score (Do NOT pass a bare integer as the 3rd argument.)
  - After any CALL ... YIELD, you MUST finish with a RETURN clause. Example:
      CALL ... YIELD node, score RETURN node, score
  - Otherwise use MATCH with WHERE + indexed properties.

  Your task:
  - Write a SINGLE valid Cypher query.
  - Output ONLY Cypher. No explanations.

  Cypher syntax rules (important):
  - Do NOT use exists(node.property).
  - Neo4j 5+ requires property existence checks to use: node.property IS NOT
      NULL
  - Always use "IS NOT NULL" instead of "exists(...)"
  - NEVER use EXPLAIN or PROFILE. Always output an executable query that ends
      with RETURN.
  - You MAY start with CALL db.index.fulltext.queryNodes(...) or CALL db.index.
      vector.queryNodes(...).
";

const userPrompt = "
  Natural language request:
  ${nlPrompt}
  Schema:
  ${schema}
";

const completion = await openai.chat.completions.create({
    model: "gpt-5", // or "gpt-5-nano"
    messages: [
      { role: "system", content: systemPrompt },
      { role: "user", content: userPrompt },
    ],
  });

const cypher = completion.choices?.[0]?.message?.content?.trim();
  if (!cypher) throw new Error("LLM returned no Cypher");
  return cypher;
}
```

B.3 TRANSFORMATION OF XML INTO A NEO4J KNOWLEDGE GRAPH

This snippet contains the Cypher-based transformation logic for importing ArchiMate XML data into Neo4j using APOC procedures. Architectural elements are mapped to nodes with both generic and type-specific labels, while relationships are reconstructed based on source–target references and dynamically assigned types. This process converts externally modeled enterprise architecture data into a structured and queryable knowledge graph.

```
export const ELEMENTS_CYPHER = "
  CALL apoc.load.xml(
  $file,
  "//*[local-name()='elements']/*[local-name()='element']",
   {},
  true
  ) YIELD value AS el
  WITH
  el.identifier AS id,
  el['xsi:type'] AS archiType,
  head([c IN coalesce(el._element, []) WHERE c._type = 'name' | c._text]) AS
      name,
  head([c IN coalesce(el._element, []) WHERE c._type = 'documentation' | c._text
      ]) AS documentation
  WHERE id IS NOT NULL
  MERGE (n:ArchiElement {id: id})
  SET n.name = name,
    n.documentation = documentation,
    n.archiType = archiType
  WITH n, archiType
  WHERE archiType IS NOT NULL
  CALL apoc.create.addLabels(n, [archiType]) YIELD node
  RETURN count(*) AS elementsImported;
";

export const RELS_CYPHER = "
  CALL apoc.load.xml(
    $file,
    "//*[local-name()='relationships']/*[local-name()='relationship']",
    {},
    true
  ) YIELD value AS r
  WITH
    r.identifier AS rid,
    r.source AS sid,
    r.target AS tid,
    r['xsi:type'] AS archiRelType,
    head([c IN coalesce(r._element, []) WHERE c._type = 'name' | c._text]) AS
        relName
  WHERE rid IS NOT NULL AND sid IS NOT NULL AND tid IS NOT NULL
  MATCH (s:ArchiElement {id: sid})
  MATCH (t:ArchiElement {id: tid})
  WITH s, t, rid, archiRelType, relName,
       toUpper(apoc.text.regreplace(coalesce(archiRelType, "ARCHI_REL"), "[^A-Za
           -z0-9_]", "_")) AS relType
  CALL apoc.merge.relationship(
    s,
```

```
    relType,
    {id: rid},
    {archiType: archiRelType, name: relName},
    t,
  {}
  ) YIELD rel
  RETURN count(*) AS relationshipsImported;
";
```

B.4 LLM-BASED GENERATION OF CONTEXTUALIZED NATURAL LANGUAGE RESPONSES

This snippet illustrates how the system generates the final user-facing response by combining the original user query, the executed Cypher statement, and the retrieved result set. The LLM is instructed to act as an experienced enterprise architect, providing concise and context-aware explanations grounded in the query results, thereby translating structured graph data into meaningful insights for the user.

```
async function explainResult(
  userPrompt: string,
  cypher: string,
  rows: unknown,
) {
  const systemPrompt = "
You are a senior enterprise architect mentoring a junior enterprise architect.
You are given:
- The junior's original question.
- The Cypher query that was executed against the EA knowledge graph.
- The resulting rows from the query.

Your task:
- Provide a concise, helpful explanation. Usually within 1  6  sentences. If a
    question is straightforward to answer, then a brief answer is appropriate,
    but it may be more if the request is complex.
- Speak like an experienced EA explaining the findings to a junior colleague.
- Do not mention the Cypher, queries, or other technical details unless relevant
     to the explanation.
- Refer to concrete numbers or entities from the result when relevant.
- If the result set is empty, explain that clearly and suggest what might be
    missing or how to refine the question.
- Respond in GitHub-flavored Markdown.
- Use formatting where it adds value, e.g.:
  - Short headings (## Summary, ## Details)
  - Bullet lists for key points
  - Fenced code blocks for Cypher or JSON snippets when helpful.
";

const userContent = "
  Original question:
  ${userPrompt}

  Executed Cypher:
  ${cypher}
```

```
  Result rows (JSON):
  ${JSON.stringify(rows, null, 2)}
";

const completion = await openai.chat.completions.create({
  model: "gpt-4.1",
    messages: [
      { role: "system", content: systemPrompt },
      { role: "user", content: userContent },
    ],
  });

const answer = completion.choices?.[0]?.message?.content?.trim();
  if (!answer) throw new Error("LLM returned no answer");

  return answer;
}
```

C

MASUTĀ EXAMPLE QUESTIONS AND ANSWERS

This chapter contains logs of example prompts and the system's responses. It serves as a showcase of how Masutā works. The examples contain the raw responses, returned by Masutā in Markdown and Cypher. The examples span three datasources: the examination office's dataset, the textbook information, and the SpeedParcel example dataset.

C.1 WHAT DOES HIS DO?

A basic question about the examination office's dataset, which Masutā is able to answer based on the contents of the knowledge graph.

```
User: What does HIS do?

Agent:
## Summary

Based on the findings, "HIS-POS" is an application component (and also
    categorized as an ArchiElement) that supports several academic and
    administrative functions.

## Details

HIS-POS enables or realizes the following application functions:

- Creating and editing module exams
- Viewing and searching student data
- Managing exam booking and grades (including entering, changing, and
    viewing grades)
- Booking EN ("Ergebnis notwendig") and NB ("Nicht bestanden") statuses

No associated business functions are explicitly documented in this dataset,
    but the supported application functions indicate that HIS-POS is likely
     integral to exam and grade management within an academic institution.
```

Listing C.1: Asking Masutā what the application HIS is.

```
MATCH (n)
WHERE (n:ApplicationComponent OR n:ApplicationFunction OR n:ArchiElement OR
    n:BusinessFunction)
  AND n.name IS NOT NULL
  AND toUpper(n.name) CONTAINS 'HIS'
OPTIONAL MATCH (n)-[:REALIZATION]->(af:ApplicationFunction)
OPTIONAL MATCH (n)-[:SERVING]->(bf:BusinessFunction)
WITH n, collect(DISTINCT af) AS afs, collect(DISTINCT bf) AS bfs
RETURN n.name AS his, labels(n) AS labels, n.documentation AS documentation,
     [a IN afs | a.name] AS realizes_application_functions, [b IN bfs | {
    name: b.name, documentation: b.documentation}] AS
    serves_business_functions
```

```
ORDER BY his
```

Listing C.2: The Cypher used to retrieve information on the HIS application from the examination office's knowledge graph.

C.2 WHAT IS AN ORPHAN?

A textbook question that generated many insights on how to interact with Masutā to get the best results. Prepending the question with the hint to stay within the realm of EAM triggered the LLM to explicitly query the information from the textbook, rather than returning a generic answer from the LLM's pretrained data.

```
User: In Enterprise architecture management, what is an orphan?
Agent:
## Summary

In enterprise architecture, an "orphan" refers to an application that does
    not support any business capability and therefore has no business value
    .

## Details

- Orphan applications are disconnected from business needs and are typically
     candidates for rationalization or retirement.
- Identifying orphans is a key activity when analyzing the application
    landscape to ensure alignment between IT assets and business value.
- In business support matrices, orphans are highlighted as not contributing
    to any required capabilities.
```

Listing C.3: Asking Masutā what an orphan is in the realm of EAM.

```
CALL db.index.fulltext.queryNodes("kg_fulltext_main", "orphan OR \"orphan
    node\" OR orphaned OR \"what is an orphan\"", {limit: 10}) YIELD node,
    score
WITH node, score, coalesce(node.definition, node.title, node.text, node.
    table_summary) AS content
OPTIONAL MATCH (sec:Section)-[:HAS_DOCUMENT]->(doc:Document)
WHERE sec = node
RETURN labels(node) AS labels, content, doc.url AS source, score
ORDER BY score DESC
```

Listing C.4: The Cypher used to query textbook information containing the keyword "orphan".

C.3 WHICH APPLICATIONS SUPPORTS THE MOST CAPABILITIES?

A question regarding the SpeedParcel dataset, which shows how Masutā is able to fit the Cypher to the dataset and fulfill the requested task.

```
User: Which application supports the most capabilities?
```

```
Agent:
## Summary

The application supporting the most capabilities is **Power BI**, with
    support for **19 distinct capabilities**.

## Details

- **Power BI** is the application that currently has the broadest capability
     coverage in your portfolio.
- This suggests it plays a significant role in enabling business functions
    and may be a key candidate for further investment or rationalization
    analysis.
```

Listing C.5: Asking Masutā which application in SpeedParcel's dataset supports the most capabilities.

```
MATCH (a:Application)
OPTIONAL MATCH (a)-[:SUPPORTS]->(c:Capability)
WITH a, count(DISTINCT c) AS capabilityCount
ORDER BY capabilityCount DESC, a.name ASC
LIMIT 1
RETURN a.name AS application, capabilityCount
```

Listing C.6: The Cypher used to count the amount of capabilities an application supports".